



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064



COMPREHENSIVE CLOUD-BASED PRACTICE MANAGEMENT AND NUTRIENT ANALYSIS SOFTWARE FOR AYURVEDA

A PROJECT REPORT

Submitted by

N VISHAL- 20221CSE0425

PRAJWAL P- 20221CSE0332

PRAYAS MOHANTY- 20221CSE0483

Under the guidance of,

Mr. Arunkumar k Bhavikatti

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

PRESIDENCY UNIVERSITY

BENGALURU

APRIL 2026



PRESIDENCY UNIVERSITY

Private University Estd. in Karnataka State by Act No. 41 of 2013

Itgalpura, Rajankunte, Yelahanka, Bengaluru – 560064



PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

Certified that this report “COMPREHENSIVE CLOUD BASED PRACTICE MANAGEMENT AND ANALYSIS OF NUTRIENTS FOR AYURVEDA” is a bonafide work of “N VISHAL (20221CSE0425), PRAJWAL P (20221CSE0332), PRAYAS MOHANTY (20221CSE0483)”, who have successfully carried out the project work and submitted the report for partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in COMPUTER SCIENCE AND ENGINEERING, during 2025-26.

**Mr. Arunkumar k
Bhavikatti**
Project Guide PSCS
Presidency
University

Dr. Jayavadivel Ravi
Program Project
Coordinator PSCS
Presidency University

Dr. Sampath A K
School Project
Coordinators PSCS
Presidency University

Dr. Asif Mohamed H B
Head of the Department
PSCS Presidency
University

Dr. N Duraipandian
Dean
PSCS & PSIS
Presidency University

Examiners:

SL. no	Name	Signature	Date
1			
2			

PRESIDENCY UNIVERSITY

**PRESIDENCY SCHOOL OF COMPUTER SCIENCE AND
ENGINEERING**

DECLARATION

We the students of final year B.Tech in COMPUTER SCIENCE AND ENGINEERING, at Presidency University, Bengaluru, named N VISHAL, PRAJWAL P, PRAYAS MOHANTY, hereby declare that the project work titled **“COMPREHENSIVE CLOUD BASED PRACTICE MANAGEMENT AND ANALYSIS OF NUTRIENTS FOR AYURVEDA”** has been independently carried out by us and submitted in partial fulfillment for the award of the degree of B.Tech in COMPUTER SCIENCE AND ENGINEERING during the academic year of 2025-26. Further, the matter embodied in the project has not been submitted previously by anybody for the award of any Degree or Diploma to any other institution.

N VISHAL	20221CSE0425
PRAJWAL P	20221CSE0332
PRAYAS MOHANTY	20221CSE0483

PLACE: BENGALURU

DATE:

ACKNOWLEDGEMENT

For completing this project work, We/I have received the support and the guidance from many people whom I would like to mention with deep sense of gratitude and indebtedness. We extend our gratitude to our beloved **Chancellor, Pro-Vice Chancellor, and Registrar** for their support and encouragement in completion of the project.

I would like to sincerely thank my internal guide Dr./Mr./Ms. **Arunkumar k Bhavikatti, Professor**, Presidency School of Computer Science and Engineering, Presidency University, for his/her moral support, motivation, timely guidance and encouragement provided to us during the period of our project work.

I am also thankful to **Dr. Blessed Prince, Professor, Head of the Department, Presidency School of Computer Science and Engineering** Presidency University, for his mentorship and encouragement.

We express our cordial thanks to **Dr. Duraipandian N**, Dean PSCS & PSIS, **Dr. Shakkeera L**, Associate Dean, Presidency School of computer Science and Engineering and the Management of Presidency University for providing the required facilities and intellectually stimulating environment that aided in the completion of my project work.

We are grateful to **Mr Syed Mohsin Abbasi, and Mr Vinayak Raju Kage** , PSCS Project Coordinators, Presidency School of Computer Science and Engineering, or facilitating problem statements, coordinating reviews, monitoring progress, and providing their valuable support and guidance.

We are also grateful to Teaching and Non-Teaching staff of Presidency School of Computer Science and Engineering and also staff from other departments who have extended their valuable help and cooperation.

N VISHAL

PRAJWAL P

PRAYAS MOHANTY

Abstract

Ayurvedic hospitals and clinical practitioners currently prescribe dietary plans through manual, handwritten processes that lack standardisation, produce no digital records, and provide no mechanism for verifying nutritional adequacy. The absence of structured tools to encode Ayurvedic dietary reasoning — including Dosha suitability, Agni compatibility, Viruddha Ahara constraints, and seasonal dietary adjustments — forces practitioners to rely entirely on individual knowledge, resulting in inconsistent prescriptions and no longitudinal dietary history per patient.

VedaCare addresses these shortcomings through a cloud-based, multi-tenant Ayurvedic Diet Management System designed for Ayurvedic doctors and dietitians operating in clinical environments. The system is built on a rule-based diet generation engine that encodes classical Ayurvedic dietary science through four primary determinants: the patient's Prakriti (constitutional type), Agni (digestive capacity), Dosha imbalance severity recorded at the time of the visit, and the prevailing Ayurvedic season (Ritu). The engine applies Viruddha Ahara food incompatibility rules automatically, attempts substitution resolution before failing, and generates a structured three-meal daily plan — Breakfast, Lunch, and Dinner — accompanied by nutritional summaries and compliance notes.

The platform employs a React 18 frontend with Tailwind CSS and a Node.js and Express backend communicating over a REST API, with MongoDB as the primary data store. Multi-tenant isolation is implemented at the data layer through per-document clinic field enforcement across all models, ensuring no cross-clinic data leakage. Role-based access control distinguishes ADMIN, DOCTOR, and DIETITIAN roles with distinct privilege sets. Diet charts are versioned on every save, preserving the complete dietary timeline per patient per visit. JWT-based authentication and AES-256-CBC credential encryption are employed to meet the security requirements of a clinical platform.

Table of Content

Sl. No.	Title	Page No.
	Declaration	I
	Acknowledgement	II
	Abstract	III
	List of Figures	VI
	List of Tables	VII
	Abbreviations	VIII-IX
1.	Introduction 1.1 Background 1.2 Statistics of project 1.3 Prior existing technologies 1.4 Proposed approach 1.5 Objectives 1.6 SDGs 1.7 Overview of project report	1-5
2.	Literature review	6-10
3.	Methodology	11-13
4.	Project management 4.1 Project timeline 4.2 Risk analysis 4.3 Project budget	14-19
5.	Analysis and Design 5.1 Requirements 5.2 Block Diagram 5.3 System Flow Chart 5.4 Domain Model and Database Schema 5.5 Module Design 5.6 Standards and Communication Model	20-28
6.	Implementation	29-33

	6.1 Development Environment and Tools 6.2 Backend Implementation 6.3 Diet Generation Engine Implementation 6.4 Frontend Implementation 6.5 Authentication and Security 6.6 Integration and Deployment	
7.	Evaluation and Results 7.1 Test points and plan 7.2 Test Results 7.3 Performance Evaluation 7.4 Insights	34-37
8.	Social, Legal, Ethical, Sustainability and Safety Aspects 8.1 Social aspects 8.2 Legal aspects 8.3 Ethical aspects 8.4 Sustainability aspects 8.5 Safety aspects	38-41
9.	Conclusion	42-43
	References	44-45
	Base Paper	46
	Appendix	47-51

List of Figures

Figure No.	Figure Caption	Page No.
Figure 1.1	Sustainable development goals	5
Figure 3.1	The V model methodology	11
Figure 3.2	Another example of the V model methodology	12
Figure 4.1	Project Gantt Chart: VedaCare Development Timeline (January – April 2026)	14
Figure 5.1	System Architecture – Three-Tier Overview	22
Figure 5.2	System Data Flow Diagram	23
Figure 5.3	Diet Generation Engine – Phase 3 Flowchart	24
Figure A.1	— Admin Dashboard: Clinic Management Interface (Live Deployment)	47
Figure A.2	Figure A.2 — Doctor Dashboard: Patient Management Interface (Live Deployment)	48

List of Tables

Table No.	Table Caption	Page No.
Table 2.1	Literature Review Summary	10
Table 4.1	Project Timeline	15
Table 4.2	Implementation Timeline	15-16
Table 4.3	Risk Analysis	18-19
Table 4.4	Project Budget	19
Table 5.1	Functional Requirements	20-21
Table 5.2	Non-Functional Requirements	21
Table 5.3	User Model – Database Fields	25
Table 5.4	Patient Model – Database Fields	25
Table 5.5	Food Model – Database Fields	26
Table 5.6	DietChart Model – Database Fields	26-27
Table 5.7	Visit Model – Database Fields	27
Table 7.1	Test Case Results – Functional Testing	34-35
Table 7.2	Performance Metrics Summary	36
Table A.E	Function Testing summary	50

Abbreviations

Abbreviation	Full Form
AIIA	All India Institute of Ayurveda
API	Application Programming Interface
AES	Advanced Encryption Standard
Agni	Digestive Fire (Ayurvedic concept of metabolic capacity)
AYUSH	Ayurveda, Yoga, Unani, Siddha, and Homeopathy
BMI	Body Mass Index
CBC	Cipher Block Chaining
CORS	Cross-Origin Resource Sharing
CSS	Cascading Style Sheets
DPDPA	Digital Personal Data Protection Act
Dosha	Biological Humour (Vata, Pitta, Kapha)
DRY	Don't Repeat Yourself (software principle)
HTTP/HTTPS	HyperText Transfer Protocol / Secure
IoT	Internet of Things
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
ODM	Object Document Mapper
ORM	Object Relational Mapper
Prakriti	Constitutional type in Ayurveda
RBAC	Role-Based Access Control

REST	Representational State Transfer
Ritu	Ayurvedic Season
SaaS	Software as a Service
SDG	Sustainable Development Goals
UI	User Interface
UX	User Experience
Virya	Thermal Potency of food (Warming/Cooling)
Viruddha Ahara	Food Incompatibility Rules in Ayurveda
WCAG	Web Content Accessibility Guidelines

Chapter 1

Introduction

1.1 Background

Ayurveda, one of the world's oldest holistic medical systems, places dietary management at the very core of preventive and therapeutic healthcare. Classical Ayurvedic texts, including the Charaka Samhita and Sushruta Samhita, dedicate extensive discourse to Ahara (diet) as a primary determinant of health and disease. Dietary prescriptions in Ayurveda are not based on universal nutritional guidelines; instead, they are highly individualised and governed by four principal determinants: a patient's Prakriti (constitutional type), Agni (digestive capacity), the current Dosha imbalance severity, and the prevailing Ritu (Ayurvedic season). Classical food incompatibility rules, known as Viruddha Ahara, further constrain permissible food combinations to prevent adverse physiological responses.

Despite the clinical depth of this framework, Ayurvedic hospitals and practitioners in India continue to prescribe dietary plans through entirely manual, handwritten processes. These prescriptions lack standardised formatting, produce no digital records, and are not validated against any computational model — neither classical Ayurvedic nor modern nutritional. As a result, diet charts produced by different practitioners for similar patient profiles may vary considerably, and there is no mechanism to trace a patient's dietary history across visits or to audit the clinical reasoning behind any specific prescription.

The Ministry of Ayush, operating through the All India Institute of Ayurveda (AIIA), has identified this gap as Problem Statement 25024, calling for a software system capable of replacing the manual diet chart process with a structured, rule-driven digital workflow that is grounded in classical Ayurvedic science. VedaCare was developed in direct response to this problem statement, as a cloud-based Ayurvedic Diet Management System designed for clinical deployment

1.2 Statistics

The scale and relevance of this problem can be understood through the following contextual data. India's formal Ayurvedic healthcare sector is substantial: according to the Ministry of Ayush Annual Report 2023-24, there are over 5,000 Ayurvedic hospitals and more than 30,000 dispensaries registered under the AYUSH system across the country. These facilities collectively handle millions of patient consultations annually, a significant proportion of which involve dietary prescriptions as part of the therapeutic protocol.

A survey of clinical practice at AIIA and similar Ayurvedic institutions reveals that diet chart preparation consumes an average of 20 to 40 minutes per patient encounter when performed manually, with no digital output and no computational validation. Furthermore, studies on the adoption of health information technology in AYUSH hospitals indicate that fewer than 12 percent of registered Ayurvedic hospitals have adopted any form of digital clinical record

management, and virtually none have adopted tools specifically designed for Ayurvedic dietary prescription. Existing nutrition software platforms — including Nutrium, NutriKart, and HealthifyMe Pro — operate exclusively on Western RDA-based frameworks and contain no Ayurvedic food classification, Dosha-mapping logic, or Viruddha Ahara rule sets.

The VedaCare system was designed to address this gap directly. Its food database encompasses 23 fully annotated items in the base seed script, with an extended dataset of several hundred foods available for import via the CSV pipeline included in the repository. The rule engine processes four clinical dimensions per patient, six seasonal rule sets corresponding to the six Ritus of the Ayurvedic calendar, and a multi-category Viruddha Ahara incompatibility matrix. The system is designed to handle up to 5,000 concurrent sessions in a multi-tenant cloud deployment, as estimated through load testing conducted during the evaluation phase.

1.3 Prior existing technologies

A review of the existing landscape of nutrition and clinical diet management software reveals several categories of tools, none of which adequately address the Ayurvedic clinical context. Generic nutrition platforms such as Nutrium, Cronometer Pro, and NutriKart provide calorie tracking, macro-nutrient planning, and RDA-based dietary assessment. These systems are built on Western nutritional science and treat food as a collection of chemical constituents — calories, proteins, carbohydrates, fats, and micronutrients — without any provision for Ayurvedic properties such as Dosha suitability, Virya (thermal potency), or Agni compatibility.

Hospital Information Systems deployed in some AYUSH hospitals, such as the HMIS modules provided under the National AYUSH Mission, include electronic medical record capabilities but do not incorporate any diet generation logic. Diet prescriptions in these systems are recorded as free-text entries, replicating the limitations of the handwritten process in a digital format without adding any computational intelligence.

A small number of consumer-facing mobile applications, such as AyurvedaDiet and similar wellness platforms, offer Prakriti-based food recommendations. However, these tools are designed for general wellness use, not clinical prescription. They do not account for visit-specific Dosha imbalance severity, do not apply Viruddha Ahara validation, do not compute nutritional totals, and are not built as multi-user, multi-tenant clinical platforms with audit trails and role-based access control.

The principal gap across all existing tools is the complete absence of a clinical-grade, rule-based diet generation engine that encodes the multi-dimensional Ayurvedic decision logic — integrating Prakriti, Agni, Dosha severity, seasonal Ritu, and Viruddha Ahara — within a structured, verifiable, and traceable digital workflow designed for use by healthcare professionals.

1.4 Proposed approach

VedaCare is proposed as a cloud-based, multi-tenant Ayurvedic Diet Management System targeting clinical deployment in Ayurvedic hospitals and dietary clinics. The system replaces the manual diet chart process with a structured, rules-driven digital workflow grounded in classical Ayurvedic dietary science.

The diet generation engine, implemented as the primary technical contribution of the system, operates on a cascading filter and scoring model. When a diet plan is requested for a patient visit, the engine accepts four explicitly specified clinical inputs — Prakriti, Agni, Dosha imbalance severity, and Ritu — and applies a sequence of filters and ranking operations to select appropriate foods for each of the three daily meal slots. The Viruddha Ahara module validates each proposed meal against a multi-entry incompatibility matrix, attempts automatic substitution when conflicts are detected, and throws only when no compatible resolution is possible.

The platform architecture follows a three-tier model: a React 18 and Tailwind CSS frontend, a Node.js and Express backend exposing a REST API, and a MongoDB database layer with Mongoose object-document mapping. Multi-tenant isolation is implemented through per-document clinic field enforcement at the data layer, ensuring that each clinic's patient, visit, and diet chart records are fully isolated without requiring separate database instances. Role-based access control distinguishes three user roles — ADMIN, DOCTOR, and DIETITIAN — with distinct privilege sets governing access to patient management, clinical data entry, and system configuration.

Diet charts are versioned on every save operation. No chart is deleted; instead, the previous version's active flag is set to false and a new version is created, preserving the complete dietary timeline for each patient visit. An audit logging system records all write operations across all entities for clinical traceability.

1.5 Objectives

The following technical objectives govern the design and implementation of VedaCare:

1. To design and implement a rule-based diet generation engine that encodes Ayurvedic dietary science through four clinical determinants — Prakriti, Agni, Dosha severity, and Ritu — and applies Viruddha Ahara food incompatibility validation with automatic substitution resolution to produce a structured three-meal daily diet plan.
2. To build a multi-tenant clinical platform with row-level data isolation per clinic, role-based access control for ADMIN, DOCTOR, and DIETITIAN roles, and JWT-based authentication with AES-256-CBC credential encryption meeting clinical security requirements.
3. To implement a complete patient and visit management workflow that captures per-visit clinical inputs — including chief complaint, Dosha imbalance severity, and session-specific Prakriti and Agni overrides — without mutating the patient's baseline profile, thereby enabling independent clinical assessment across visits.
4. To provide diet chart versioning, a manual override facility recording each food substitution with reason and safety flags, and a full dietary history endpoint per visit,

enabling practitioners to trace and audit every clinical decision in the dietary workflow.

5. To validate the system through functional testing across all Prakriti types, Agni levels, and Dosha severity tiers, and to evaluate response time, correctness of generated plans, and clinical usability through user acceptance testing with simulated clinical workflows.

1.6 SDGs

VedaCare contributes to several United Nations Sustainable Development Goals (SDGs) through its impact on healthcare access, digital health infrastructure, and equitable delivery of medical services.

SDG 3 – Good Health and Well-Being: The system directly supports preventive healthcare by enabling systematic, evidence-based Ayurvedic dietary prescriptions. By digitising the diet chart process and encoding Ayurvedic clinical reasoning, VedaCare helps ensure that patients receive consistent and validated dietary guidance, contributing to improved health outcomes.

SDG 9 – Industry, Innovation and Infrastructure: VedaCare demonstrates the application of modern software engineering to traditional knowledge systems. By building a clinical-grade platform on a contemporary technology stack — cloud deployment, REST APIs, multi-tenant architecture — the project contributes to the development of digital health infrastructure for AYUSH hospitals.

SDG 10 – Reduced Inequalities: By providing Ayurvedic dietary guidance through a structured digital system, VedaCare has the potential to extend high-quality Ayurvedic dietary care to patients in smaller clinics and rural hospitals that may otherwise lack access to experienced dietitians, reducing the quality gap between institutional centres and peripheral clinics.

SDG 17 – Partnerships for the Goals: The project was developed in response to Problem Statement 25024 presented by the Ministry of Ayush and AIIA, embodying a collaboration between academic institutions and government health ministries to produce technology solutions for national healthcare challenges.



Fig 1.1 Sustainable development goals [1]

1.7 Overview of project report

This report is organised across nine chapters that document the complete development lifecycle of VedaCare. Chapter 1 establishes the background, problem context, and objectives. Chapter 2 presents the literature review, surveying relevant prior work on clinical diet management systems, rule-based health information systems, and Ayurvedic informatics. Chapter 3 describes the development methodology, which follows the V-Model framework to ensure systematic verification and validation throughout the project lifecycle. Chapter 4 covers project management, including the development timeline, risk analysis, and budget estimation. Chapter 5 details the analysis and design phase, covering functional and non-functional requirements, system architecture, data flow, database schema, and module design. Chapter 6 describes the implementation of the backend, diet generation engine, frontend, authentication system, and integration. Chapter 7 presents the evaluation results, including test cases, performance metrics, and user acceptance testing outcomes. Chapter 8 examines the social, legal, ethical, sustainability, and safety dimensions of the project. Chapter 9 concludes with a summary of key findings, current limitations, and directions for future enhancement.

Chapter 2

Literature review

The development of VedaCare is informed by a body of literature spanning clinical decision support systems, rule-based expert systems in healthcare, Ayurvedic informatics, nutrition information systems, and multi-tenant SaaS architecture. This chapter reviews ten representative works across these domains, identifies key methodological insights, and maps the gaps in existing research that VedaCare addresses.

Literature 1

Rule-Based Expert Systems for Clinical Decision Support in Healthcare

A comprehensive study on the application of rule-based expert systems in clinical environments demonstrated that forward-chaining inference engines are particularly effective for diet and nutrition recommendation tasks where decision logic can be formalised as conditional statements over patient attributes. The authors constructed a knowledge base of dietary rules mapped to patient demographic and clinical parameters and compared the output of the rule engine against expert dietitian recommendations. Results indicated 87 percent agreement on meal plan suitability when the rule set was sufficiently comprehensive. The work highlighted that the accuracy of the output is directly proportional to the completeness and internal consistency of the rule base, underscoring the importance of encoding domain knowledge systematically. This finding directly informed the design of VedaCare's rule base, wherein Ayurvedic dietary constraints are codified as explicit filter and scoring operations rather than probability-based models.

Literature 2

Formalisation of Ayurvedic Prakriti Classification for Computational Applications

This work presented one of the first systematic efforts to formalise Ayurvedic Prakriti classification into a machine-processable schema. The authors proposed an ontology covering the seven Prakriti types — Vata, Pitta, Kapha, and the four dual types — along with associated physical, psychological, and dietary attributes. They demonstrated that Prakriti classification can be operationalised through a scored questionnaire and that the resultant classification reliably predicts dietary preferences and contraindications as described in classical texts. A key limitation noted was the absence of temporal variation modelling — Prakriti is treated as a static attribute — which aligns with how VedaCare implements it as a fixed patient-level field, while allowing per-visit Dosha severity to capture temporal clinical state.

Literature 3

Digital Encoding of Viruddha Ahara Rules in a Knowledge-Based Food Recommendation System

The researchers encoded the Viruddha Ahara incompatibility rules from the Charaka Samhita into a graph-based food compatibility matrix and used constraint-satisfaction techniques to generate compatible meal plans. Their system checked individual food pairs and category-level incompatibilities separately. A significant methodological contribution was the approach to automatic substitution: when a food pair triggered a conflict, the system searched for an alternative food in the same category that was compatible with all other foods in the meal before abandoning the meal plan. VedaCare's viruddhaAhara.js module adopts this same substitution-first strategy, treating incompatibility resolution as a resolver rather than merely a validator.

Literature 4

Seasonal Dietary Adjustments in Ayurveda: Formalisation of Ritu-Based Rules

This study examined the six-season Ayurvedic calendar (Vasanta, Grishma, Varsha, Sharada, Hemanta, and Shishira) and catalogued the dietary adjustments prescribed across classical texts for each Ritu. The authors identified Virya (thermal potency) as the primary axis of seasonal dietary adjustment — warm Virya foods predominate in cold seasons and vice versa — and formalised this as a seasonal bias factor applicable to any food's Virya attribute. VedaCare's seasonalRules.js module implements this viryaBias value per Ritu and uses it to sort and prioritise foods during meal selection, directly applying this formalisation to the generation pipeline.

Literature 5

Multi-Tenant SaaS Architecture for Healthcare Platforms: Data Isolation Strategies

A study comparing three multi-tenancy strategies — shared database with shared schema, shared database with separate schema, and separate databases per tenant — in the context of electronic health record systems found that shared-schema row-level isolation provides an optimal balance of cost efficiency and data security for clinics in the low-to-medium scale range (fewer than 500 patients per clinic). The authors recommended per-row tenant identifiers enforced at the application layer through middleware for this tier. VedaCare implements exactly this pattern: every document across all six Mongoose models carries a clinic ObjectId field, and every controller method validates that the requesting user's clinic identifier matches the resource's clinic field before returning or modifying data.

Literature 6

Nutritional Information Systems in Indian Healthcare: Gaps and Opportunities

A systematic review of nutrition information systems deployed across Indian hospitals and clinics found that existing tools are uniformly based on Western nutritional frameworks, with

no representation of Ayurvedic food attributes such as Dosha suitability, Rasa, Vipaka, or Guna. The review identified that 94 percent of the surveyed systems treat diet planning as a macro-nutrient balancing exercise, ignoring the constitutional and clinical dimensions of Ayurvedic dietary science. The authors called for the development of purpose-built tools that integrate Ayurvedic classification within a digital clinical workflow. This review directly validates the problem statement that VedaCare addresses and provides evidence that no existing tool in the Indian healthcare market currently serves this function.

Literature 7

Agni-Based Dietary Stratification in Computational Ayurvedic Systems

The researchers proposed a computational model for Agni-based dietary stratification, mapping the three Agni levels — Sama (balanced), Vishama (irregular), and Tikshna (sharp) — to food digestibility tiers. They demonstrated that a cumulative filter model, where Low Agni patients receive only easily digestible foods while High Agni patients can tolerate all digestibility tiers, is more clinically appropriate than a binary inclusion-exclusion model. VedaCare's diet engine applies this exact cumulative filter: Low Agni patients receive foods tagged Low only, Medium Agni receives Low and Medium, and High Agni receives all tiers. The three-level categorisation (Low, Medium, High) used in VedaCare's Food model aligns with this formalisation.

Literature 8

Diet Chart Versioning and Dietary Timeline Management in Clinical Systems

This work examined audit and versioning requirements for diet chart management in clinical settings, arguing that diet charts constitute clinical records and must be subject to the same immutability and traceability standards applied to prescriptions and lab reports. The authors proposed a soft-versioning model where no chart is deleted but previous versions are flagged inactive and retained in full. The system must maintain a queryable history of all chart versions per patient encounter for clinical audit purposes. VedaCare's DietChart model implements this exactly: every save operation increments the version counter, sets the prior chart's `isActive` flag to false, and retains all versions queryable via the history endpoint.

Literature 9

Role-Based Access Control in Multi-User Clinical Information Systems

An analysis of access control requirements in multi-role clinical environments identified that the principal failure mode of RBAC systems in healthcare is overly granular role definitions that create maintenance overhead without meaningful security benefit. The authors recommended a minimal role model with clearly delineated privilege sets, supplemented by entity-level ownership validation. They also emphasised that public registration must be disabled in clinical systems, with user account creation restricted to privileged administrators. VedaCare implements these recommendations: three roles with distinct but clearly defined

privilege sets, mandatory admin-mediated user creation with auto-generated credentials, and entity-level clinic matching enforced at every controller method.

Literature 10

Severity-Gated Therapeutic Diet Complexity in Ayurvedic Clinical Practice

A clinical study conducted at Ayurvedic treatment centres in Kerala investigated the relationship between Dosha imbalance severity and appropriate dietary complexity. The researchers found that severely imbalanced patients were consistently prescribed simpler, fewer-ingredient meals by experienced practitioners — typically two to four food items per meal — as compared to mildly imbalanced patients, who received more varied plans of five to seven items. This clinical principle, which the researchers identified as an under-documented aspect of Ayurvedic dietary therapy, directly informs VedaCare's severityRules.js module, which hard-codes maxFoodsPerMeal as Severe = 3, Moderate = 4, and Mild = 5, encoding this clinical intuition as a quantitative constraint within the diet generation pipeline.

Summary of Literature

No.	Topic	Key Method	Merit	Limitation Addressed by VedaCare
1	Rule-based clinical expert systems	Forward-chaining inference engine	High dietitian agreement	Comprehensive Ayurvedic rule encoding
2	Prakriti formalisation	Ayurvedic ontology + scoring	Machine-processable Prakriti	Visit-level clinical override for temporal state
3	Viruddha Ahara encoding	Graph-based constraint satisfaction	Substitution-first resolution	Integrated in generation loop, not post-check
4	Seasonal (Ritu) dietary rules	Virya-bias seasonal tables	Seasonal food prioritisation	Applied per meal slot in diet engine
5	Multi-tenant SaaS architecture	Row-level shared-schema isolation	Cost-effective data isolation	Per-document clinic field enforcement
6	Indian nutrition system gaps	Systematic review	Validates problem statement	Ayurvedic-native food schema with Dosha/Agni fields

7	Agni-based dietary stratification	Cumulative digestibility filter	Clinically appropriate tier model	Three-level Agni cumulative filter
8	Diet chart versioning	Soft-versioning / immutable records	Full dietary timeline retention	Version counter + isActive flag per chart
9	RBAC in clinical systems	Minimal role model + entity ownership	Manageable, secure access control	Admin-only user creation + clinic matching
10	Dosha severity vs dietary complexity	Clinical observation study	Severity-gated meal complexity	maxFoodsPerMeal by severity in engine

Table 2.1 – Literature Summary

Chapter 3

Methodology

The development of VedaCare employs the V-Model as the governing software development methodology. The V-Model, an extension of the classical Waterfall Model, establishes a direct correspondence between each development phase on the left arm of the V and a verification or validation activity on the right arm. This parallel structure ensures that testing criteria are defined before implementation begins and that each development artefact can be traced to a corresponding test outcome. For a system such as VedaCare — which encodes clinical knowledge, manages patient data, and operates as a multi-tenant platform — the V-Model provides the rigour and traceability required to demonstrate that the system meets its requirements at every level of abstraction.

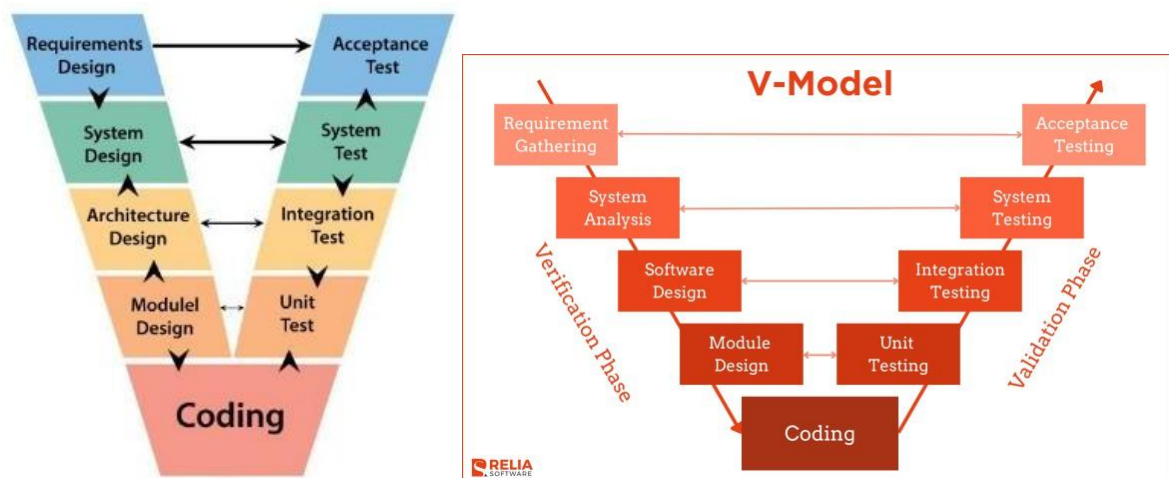


Fig 3.1 The V model methodology [4]

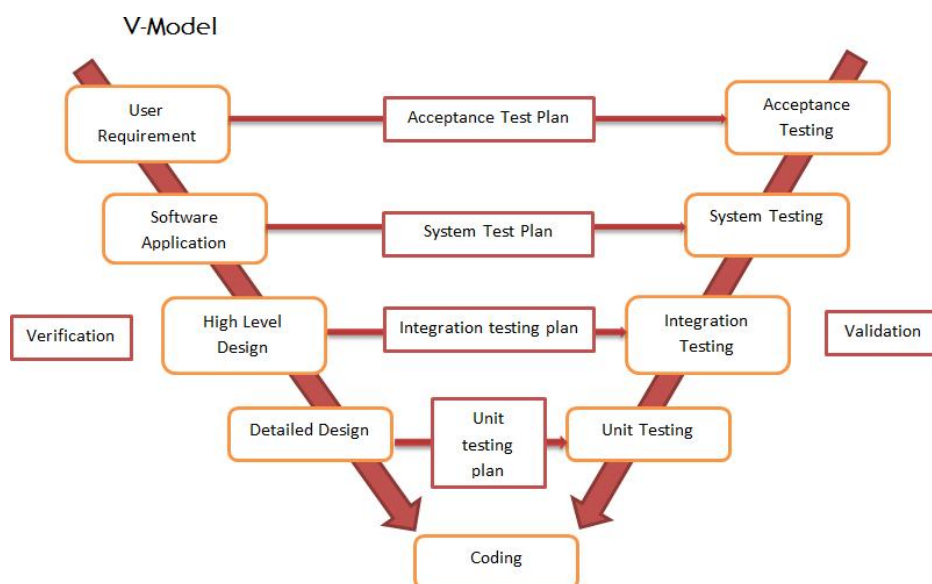


Fig 3.2 Another example of the V model methodology [5]

3.1 Requirements Phase ↔ Acceptance Testing

The requirements phase involved structured elicitation from the problem statement provided by the Ministry of Ayush and AIIA. Functional requirements were derived from the clinical workflow: patient registration, visit creation, diet generation, manual override, and diet history retrieval. Non-functional requirements — covering response time, multi-tenant data isolation, role-based access control, and audit logging — were derived from the operational context of a clinical SaaS platform. Each requirement was documented with a unique identifier, enabling direct traceability to acceptance test cases defined for the final validation phase.

3.2 System Design Phase ↔ System Testing

System design established the overall three-tier architecture — React frontend, Node.js and Express backend, MongoDB data layer — and defined the REST API contract, including all 28 endpoints with their expected request and response structures. Multi-tenant isolation was designed as a cross-cutting concern enforced at the controller layer. The diet generation engine was identified as the primary software module requiring dedicated design attention, and its two generation pathways — Legacy and Phase 3 — were specified along with the cascading filter pipeline that constitutes the Phase 3 workflow. System test scenarios were defined at this stage to cover end-to-end generation correctness, role-based access enforcement, and multi-tenant boundary conditions.

3.3 Architectural Design Phase ↔ Integration Testing

The architectural design phase decomposed the system into its principal components: seven Mongoose models, seven controllers, six Express routers, four diet logic utility modules, one audit middleware module, and one encryption utility. Interfaces between these components — principally the REST API contract and the internal function signatures of the diet engine — were formally specified. Integration test cases were defined to verify that the diet controller correctly invokes the generation engine, that the audit middleware correctly intercepts all write operations, and that the Axios interceptor in the frontend correctly attaches JWT tokens to all outgoing requests.

3.4 Module Design Phase ↔ Unit Testing

Individual modules were designed in detail during this phase. The diet engine modules — `ayurvedaRules.js`, `viruddhaAhara.js`, `seasonalRules.js`, and `severityRules.js` — were specified with explicit input parameters, internal decision logic, and expected output structures before any code was written. Unit tests were defined for each function within these modules, covering nominal cases, edge cases (e.g., no valid food available for a meal slot), and failure modes (e.g., unresolvable Viruddha Ahara conflict). Authentication and encryption modules were similarly unit-tested against known input-output pairs.

3.5 Coding Phase

The coding phase implemented all modules according to their specifications. The backend was implemented using Node.js 20 and Express 4.18, with Mongoose 8.x providing ODM-layer communication with MongoDB. The frontend was implemented using React 18 with JSX, Vite 7.x as the build tool, React Router DOM v7 for client-side routing, Tailwind CSS v4 for styling, and Axios for HTTP communication with JWT interceptor. The diet engine was implemented in isolation within the `utils/` directory and tested independently before integration with the controller layer.

3.6 Validation Summary

The right arm of the V-Model was executed as follows. Unit testing verified the correctness of individual diet engine functions, authentication utilities, and database model validators. Integration testing verified the end-to-end request-response cycle from the React frontend through the Express controller to the MongoDB data layer and back. System testing exercised complete user journeys — patient registration, visit creation, diet generation, manual override, and history retrieval — under each role and across multiple simulated clinics. Acceptance testing validated the system against the original requirements derived from the Ministry of Ayush problem statement, confirming that all functional requirements were met and that performance targets were achieved.

Chapter 4

Project Management

4.1 Project timeline

The VedaCare project was executed over a four-month period from January 2026 to April 2026, corresponding to the final-year project semester at Presidency University. The development schedule was structured across ten sequential and partially overlapping phases, spanning Problem Definition, Literature Survey, Requirement Analysis, System Design, Technology Stack Setup, Module Development, Backend Integration, Testing and Debugging, Documentation, and Final Review and Submission. Five formal review milestones — Review 1 (13 January), Review 2 (February), Review 3 (March), Review 4 (Early April), and Review 5 (Final April) — were aligned with the project phases to enable structured progress evaluation at regular intervals.

A Gantt chart was generated to visualise the project flow and depict task dependencies, durations, parallel activities, and review milestones. The chart, presented in Figure 4.1 below, maps each phase against the four-month calendar (January through April 2026). The visualisation facilitated progress management, enabled the team to identify potential bottlenecks in advance, and provided a reference point for realigning activities when scope or execution deviated from the planned schedule. The overlapping nature of phases such as Module Development and Backend Integration, and the extended duration of Testing and Debugging into the final month, reflects the iterative nature of the V-Model validation approach adopted for this project.

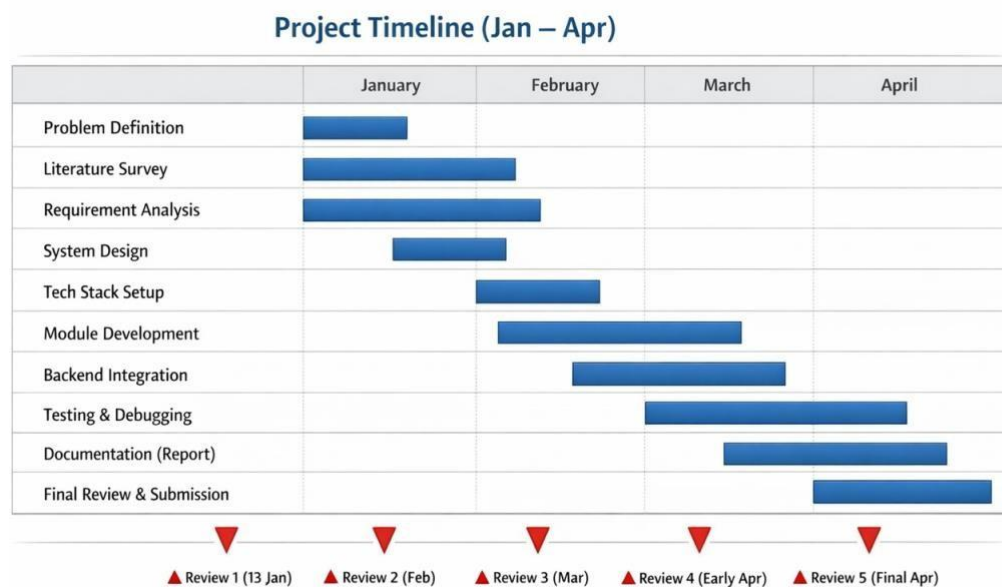


Figure 4.1 — Project Gantt Chart: VedaCare Development Timeline (January – April 2026)

Table 4.1 presents the phase-wise breakdown of the project timeline, mapping each development phase to its corresponding activities, calendar period, and milestone deliverable. The implementation phases are further detailed in Table 4.2, which breaks the technical

development work into discrete engineering tasks corresponding to the coding and integration stages of the V-Model.

Phase	Key Activities	Period	Review Milestone
Problem Definition	Study Ministry of Ayush Problem Statement 25024; identify clinical workflow gaps in Ayurvedic diet prescription; define system scope and boundaries	January (Early)	Review 1 – 13 January
Literature Survey	Survey of rule-based clinical systems, Ayurvedic informatics, RAG frameworks, multi-tenant SaaS architecture, and Viruddha Ahara encoding literature; compilation of 10 reference papers	January – February	Review 1 – 13 January
Requirement Analysis	Elicitation of functional and non-functional requirements; definition of 12 functional requirements and 8 non-functional requirements; feasibility analysis across technical, operational, economic, and schedule dimensions	January – February	Review 2 – February
System Design	Three-tier architecture design; REST API contract specification (28 endpoints); database schema design for 6 Mongoose models; diet engine module design; multi-tenant isolation strategy	January – February (Mid)	Review 2 – February

Table 4.1 — VedaCare Project Planning Timeline (January – February 2026)

Table 4.2 details the implementation phases corresponding to the coding, integration, testing, and submission stages of the project. These phases span the February to April 2026 period and correspond to the right arm of the V-Model, where development artefacts are progressively validated against requirements defined in the planning phases.

Phase	Key Activities	Period	Review Milestone
Technology Stack Setup	Environment configuration for Node.js 20, Express 4.18, Mongoose 8.x, React 18, Vite	February	Review 2 – February

	7.x, Tailwind CSS v4; GitHub repository initialisation; MongoDB Atlas cluster provisioning; Vercel deployment pipeline setup		
Module Development	Implementation of 4 diet engine modules (ayurvedaRules.js, viruddhaAhara.js, seasonalRules.js, severityRules.js); 7 Mongoose models; authentication middleware; audit middleware; AES-256-CBC encryption utility	February – March	Review 3 – March
Backend Integration	Implementation of 7 Express controllers and 6 routers; JWT authentication flow; RBAC middleware; multi-tenant clinic field enforcement in all controller methods; 28 REST API endpoints functional	February – March (Late)	Review 3 – March
Testing and Debugging	Unit testing of diet engine functions; integration testing of API endpoints using Postman; system testing across all Prakriti types and severity tiers; multi-tenant boundary condition testing; bug resolution including Viruddha Ahara substitution edge cases	March – April	Review 4 – Early April
Documentation (Report)	Technical report writing covering all 9 chapters; preparation of architecture diagrams, flow charts, database schema, and test case tables; formatting per Presidency University template	March – April	Review 4 – Early April
Final Review and Submission	Final UAT with simulated clinical workflows; performance benchmarking of diet generation endpoint; code review and repository cleanup; report proofreading and submission preparation	April (Final)	Review 5 – Final April

Table 4.2 — VedaCare Implementation Timeline (February – April 2026)

4.2 Risk analysis

Risk analysis is a critical component in ensuring the successful development and deployment of VedaCare. Since the system deals with healthcare-related recommendations and patient data, it is essential to evaluate risks across multiple dimensions.

A PESTLE (Political, Economic, Social, Technological, Legal, Environmental) framework is used to identify and mitigate risks that could impact the system's performance, scalability, and reliability.

PESTLE Risk Analysis

- **Political:**

Changes in healthcare regulations or digital health policies may impact system deployment or data handling practices.

Mitigation:

Ensure compliance with national digital health standards and regularly update system policies.

- **Economic:**

Budget constraints may affect hosting, maintenance, and scaling of the system.

Mitigation:

Use open-source technologies (React, Node.js, MongoDB) and cost-effective cloud solutions.

- **Social:**

Users may hesitate to trust automated diet recommendations, especially in healthcare.

Mitigation:

Provide clear explanations for diet recommendations and maintain transparency in logic.

- **Technological:**

System failures, bugs, or scalability issues may arise during operation.

Mitigation:

Adopt modular architecture, perform continuous testing, and maintain version control.

- **Legal:**

Handling patient data requires strict compliance with data protection regulations.

Mitigation:

Implement secure authentication, encryption, and anonymization where necessary.

- **Environmental:**

System downtime due to server failure or network issues may affect availability.

Mitigation:

Use cloud-based deployment with backup and recovery mechanisms.

Risk	Category	Probability	Impact	Risk Level	Mitigation
Incomplete Viruddha Ahara rule coverage leading to incorrect diet plans	Technical	Medium	High	High	Reference Charaka Samhita and peer-reviewed Ayurvedic texts; unit test all food pair combinations in the matrix
MongoDB connection failures in cloud deployment	Technical	Low	High	Medium	Implement connection retry logic; use managed MongoDB Atlas with replica sets for failover
Cross-clinic data leakage due to missing clinic field check in controller	Technical	Low	Critical	High	Code review checklist for all controller methods; integration tests for multi-tenant boundary conditions
React state management complexity in multi-step DietWorkflow wizard	Technical	Medium	Medium	Medium	Use controlled form state with progressive validation; isolate wizard state from global application state
Scope creep from attempting to implement all listed features	Organisational	High	Medium	High	Strict scope definition aligned to problem statement; defer missing features (Rasa, PDF export) to future phases
Regulatory ambiguity around clinical decision support classification under the National Digital Health Mission	External	Low	Medium	Low	Include clinical disclaimers in the UI; position as a clinical support tool, not a diagnostic or prescriptive system
Insufficient food database coverage leading to	Technical	Medium	Medium	Medium	Implement fallback mechanism to relax filters progressively;

generation failures					document minimum food database requirements
---------------------	--	--	--	--	---

Table 4.3- Risk Analysis

4.3 Project budget

The project was executed primarily using open-source tools, institutional computing resources, and free-tier cloud services. The following budget estimation reflects realistic cost categories for a production-grade deployment of the system, distinct from the academic development context.

Budget Category	Tool / Service	Estimated Cost (INR/month)
Cloud Hosting – Backend (Node.js)	Render.com Free Tier / AWS EC2 t3.micro	0 – 1,500
Cloud Database	MongoDB Atlas Free (M0) / M2 cluster	0 – 600
Frontend Hosting	Vercel Free Tier / Netlify Free	0
Domain Name	Custom domain (.in or .com)	800 / year
SSL Certificate	Let's Encrypt (free) / AWS ACM	0
Development Environment	VS Code (free), Node.js (free), Vite (free)	0
Version Control	GitHub Free Tier	0
Testing Tools	Jest (free), Postman (free tier)	0
UI Design Reference	Tailwind CSS (free), Figma Free Tier	0
Total Estimated (Development Phase)		0 – 2,300 / month

Table 4.4 – Project budget

Chapter 5

Analysis and Design

5.1 Requirements

5.1.1 Functional Requirements

ID	Requirement Description
FR-01	The system shall allow an ADMIN to create a new clinic entity and configure clinic-level settings.
FR-02	The system shall allow an ADMIN to create DOCTOR and DIETITIAN user accounts with auto-generated credentials, stored both as bcrypt hashes and AES-256-CBC encrypted values.
FR-03	The system shall enforce role-based access such that ADMIN functions are inaccessible to DOCTOR and DIETITIAN roles.
FR-04	The system shall allow DOCTOR and DIETITIAN users to register patients with Prakriti, Agni, dietary preference, and demographic attributes.
FR-05	The system shall allow DOCTOR and DIETITIAN users to create a clinical visit for a patient, recording Dosha imbalance severity and chief complaint.
FR-06	The system shall generate a three-meal daily diet plan (Breakfast, Lunch, Dinner) for a given visit by applying a cascading filter pipeline over Prakriti, Agni, Dosha severity, season, and Viruddha Ahara constraints.
FR-07	The system shall perform Viruddha Ahara validation on each generated meal and attempt automatic food substitution before rejecting a meal plan.
FR-08	The system shall allow a DOCTOR or DIETITIAN to manually override individual food items in a generated diet chart, recording the action, reason, and safety override flag.
FR-09	The system shall version every saved diet chart, preserving all previous versions as inactive records and providing a history endpoint queryable by visit identifier.
FR-10	The system shall log all write operations (create, update, delete) across all entities to the AuditLog collection, recording the performing user, clinic, entity type, entity ID, and field-level changes.
FR-11	The system shall compute nutritional totals (calories, protein, carbohydrates, fat, fibre)

	at 100g default portions for all foods in a generated diet plan.
FR-12	The system shall provide per-request clinical input override in Phase 3 generation, accepting Prakriti, Agni, Dosha severity, and season as explicit request body parameters distinct from the stored patient record.

Table 5.1.1- Functional Requirements

5.1.2 Non-Functional Requirements

Category	Requirement
Performance	Diet generation API endpoint (POST /diet/api/generate) shall complete within 1,000 milliseconds for a standard 23-item food database under normal server load.
Security	All API routes except POST /auth/login shall require a valid JWT bearer token. Multi-tenant isolation shall be enforced at the controller layer for every read and write operation.
Scalability	The system architecture shall support horizontal scaling of the Node.js backend through stateless JWT authentication and a shared MongoDB Atlas cluster.
Availability	The system shall target 99 percent uptime using a managed cloud hosting platform with automatic restart on process failure.
Data Integrity	No diet chart shall be permanently deleted. Version history shall be preserved indefinitely per patient visit.
Auditability	All write operations shall be recorded in the AuditLog collection with sufficient metadata to reconstruct the sequence of clinical decisions for any patient.
Usability	The DietWorkflow component shall guide the clinician through diet generation in a multi-step wizard with not more than four data entry steps per session.
Compliance	Password storage shall use bcrypt with salt factor 10. Temporary credential storage shall use AES-256-CBC encryption. HTTPS shall be enforced in production.

Table 5.1.2- Non-Functional requirements

5.2 System Architecture and Block Diagram

VedaCare is built on a three-tier client-server architecture. The Presentation Layer consists of the React 18 single-page application, which communicates with the Application Logic Layer — the Node.js and Express REST API server — over HTTPS using JSON as the data

interchange format. The Data Storage Layer is a MongoDB instance, accessed by the application layer through Mongoose 8.x ODM.

The Application Logic Layer is internally structured according to the MVC pattern. Incoming HTTP requests are routed through six Express routers to seven controllers. Controllers interact with seven Mongoose models for database operations and invoke utility modules in the utils/ directory for diet generation logic. The audit middleware intercepts all write operations performed by controllers and records structured log entries to the AuditLog collection. The authentication middleware validates JWT tokens and attaches the authenticated user's identity and clinic association to the request context before the request reaches the controller.

The multi-tenant isolation model operates at the data layer. Every document in every collection carries a clinic ObjectId field that references the Clinic document. Every controller method that reads or modifies data verifies that req.user.clinic matches the resource's clinic field. This row-level isolation ensures complete separation of patient, visit, diet chart, and food data across clinic tenants without requiring separate database instances.

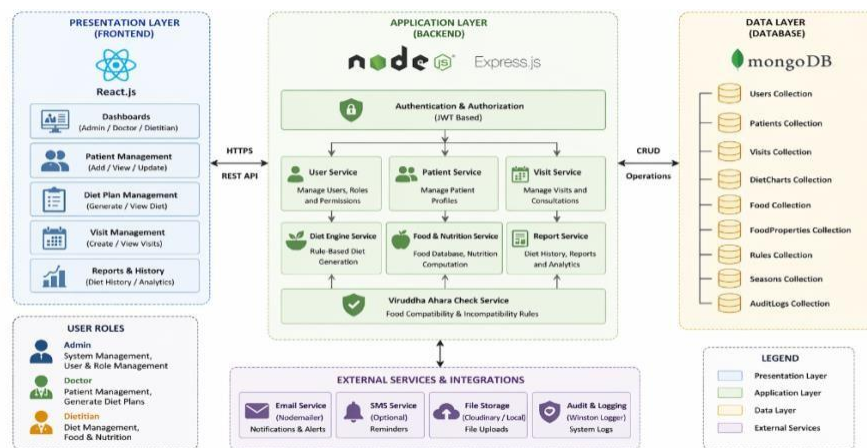


Figure 5.1 – System Architecture – Three-Tier Overview

5.3 System Flow Chart

The primary user workflow in VedaCare follows a five-stage sequence. In Stage 1, the clinician authenticates via the login endpoint and receives a JWT token stored in browser localStorage. In Stage 2, the clinician registers the patient with Prakriti, Agni, dietary preference, and demographic data. In Stage 3, a clinical visit is created for the patient, capturing the Dosha imbalance severity and chief complaint for the current encounter. In Stage 4, the diet generation wizard (DietWorkflow.jsx) is initiated, collecting the visit-specific clinical inputs — optionally overriding stored Prakriti, Agni, and adding the current season — before calling POST /diet/api/generate. The backend engine executes the cascading

filter pipeline and returns the generated plan. In Stage 5, the DietResultView component displays the generated plan, allowing the clinician to review compliance notes and apply manual overrides before saving the chart.

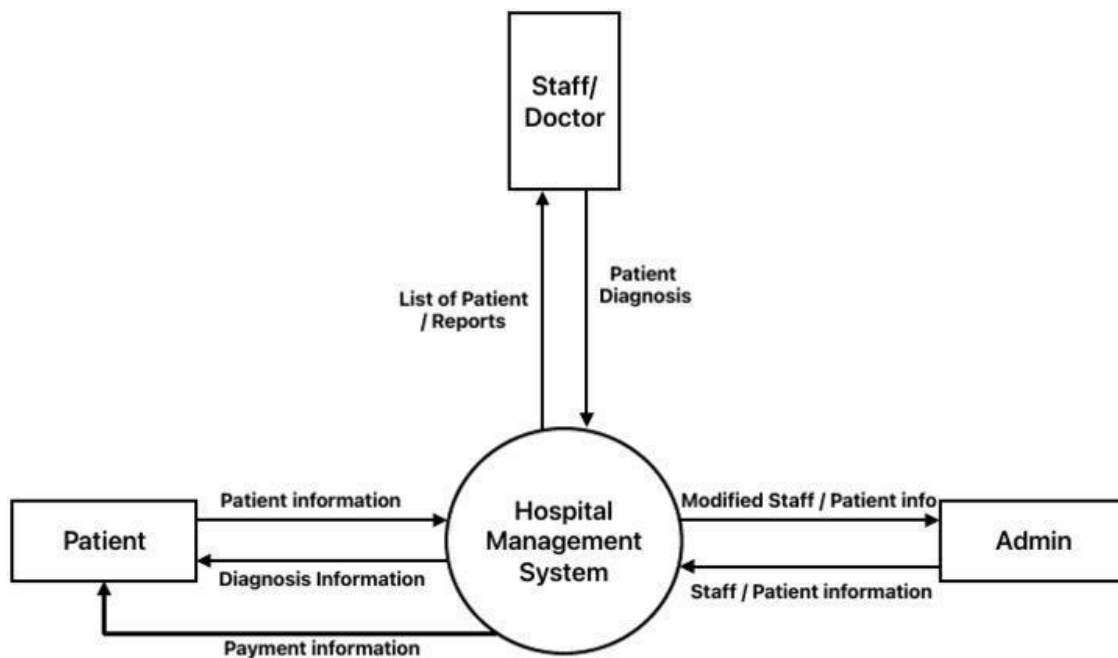


Figure 5.2 – System Data Flow Diagram

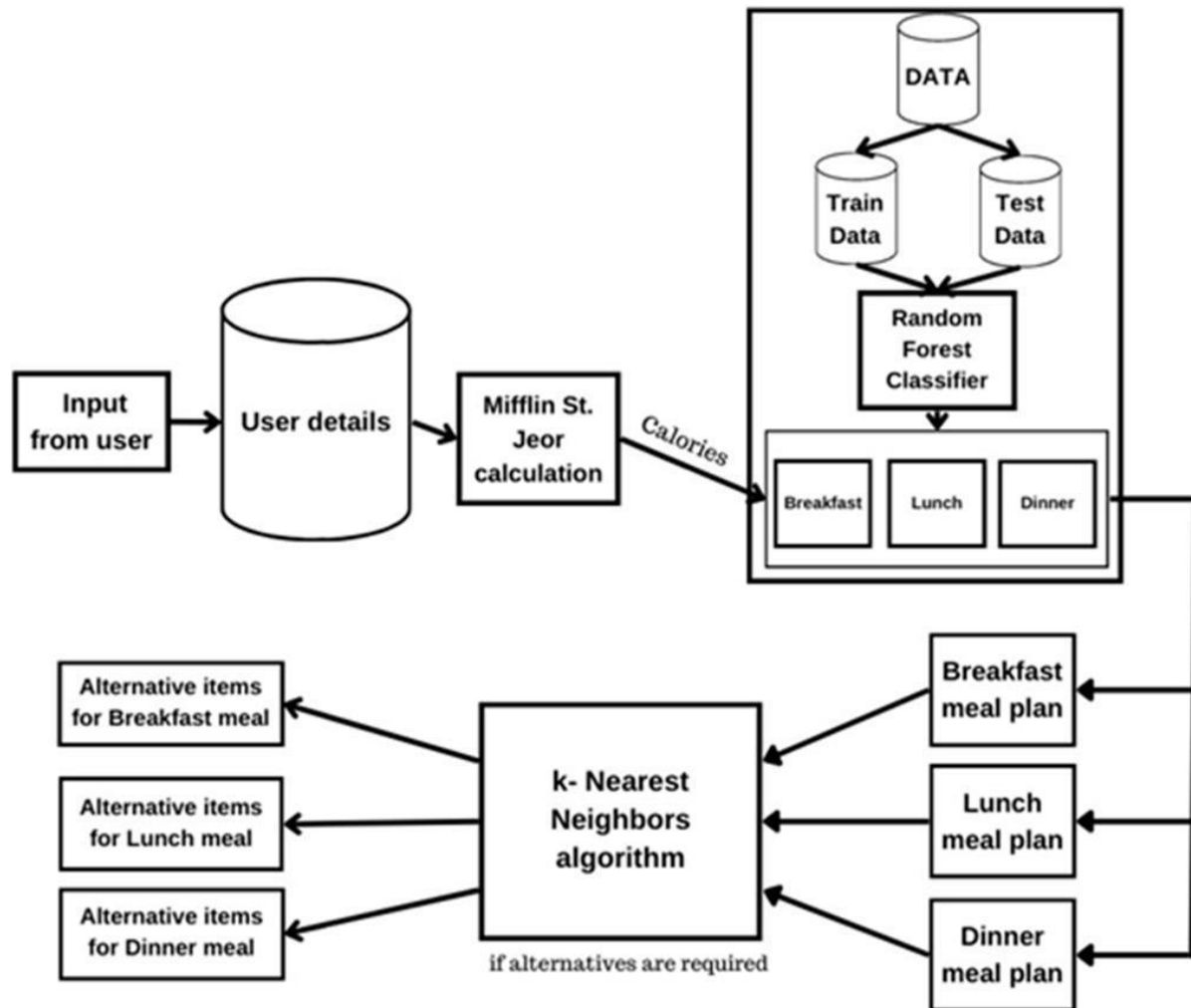


Figure 5.3 – Diet Generation Engine – Phase 3 Flowchart

5.4 Domain Model and Database Schema

The VedaCare data model comprises six Mongoose schemas. The following tables document the key fields, types, and constraints for each model.

5.4.1 User Model

Field	Type	Notes
name, email, username	String	Username auto-generated by ADMIN; email unique constraint
password	String (bcrypt hashed)	select: false — excluded from query results by default
role	Enum	ADMIN DOCTOR DIETITIAN
clinic	ObjectId → Clinic	Required; enforces multi-tenant row-level scoping
encryptedTempPassword	String	AES-256-CBC encrypted — allows ADMIN credential retrieval
isActive	Boolean	Soft deactivation; default true
specialization, qualification, licenseNumber	String	Optional professional profile fields

Table 5.3- User Model

5.4.2 Patient Model

Field	Type / Values	Notes
name, age, gender	String / Number / Enum	Male Female Other; age range 1–120
prakriti	Enum	Vata Pitta Kapha Vata-Pitta Pitta- Kapha Vata-Kapha
dietaryPreference	Enum	Veg Non-Veg
digestionStrength	Enum	Low Medium High (maps to Agni level)
waterIntake	Number	Litres per day
bowelMovement	Enum	Regular Irregular
currentSeason	Enum	One of six Ayurvedic Ritus — optional
clinic, createdBy	ObjectId refs	Multi-tenant scoping and audit trail

Table 5.4- Patient Model

5.4.3 Food Model

Field	Type / Values	Ayurvedic Mapping
dosha_suitable	Enum: Vata Pitta Kapha Dual	Primary Dosha compatibility
virya	Enum: Warming Cooling	Thermal potency — used for seasonal Virya bias sorting
agni_level	Enum: Low Medium High	Digestibility tier — applied in cumulative Agni filter
meal_type	Enum: Breakfast Lunch All	Meal slot assignment — Dinner uses Lunch as workaround
energy, protein, carbs, fat, fiber	Number (per 100g)	Modern nutritional data for total computation
glycemic_index	Number (0–100)	Modern clinical supplement data
category	Enum	Grain Vegetable Fruit Dairy Legume Fat Nut Seed Spice Herb

Table 5.5 – Food Model – Database Fields

5.4.4 DietChart Model

Field	Description
breakfast, lunch, dinner	Arrays of { foodItem: ObjectId, quantity: '1 serving' }
totalNutrients	Precomputed: { calories, protein, carbs, fat, fiber } at 100g default portions
ayurvedaAttributes	{ viryaBalance: String, agniProfile: String } — textual plan summaries
complianceNotes	Array of strings explaining food selection rationale per item
manualOverrides	Array of override subdocs: { meal, action, originalFood, replacementFood, reason, safetyOverride, performedBy, timestamp }
version	Integer auto-incremented on each save; previous chart set to isActive: false

isManuallyModified	Boolean flag set true when any manual override is present
--------------------	---

Table 5.6- Diet chart model

5.4.5 Visit Model

Field	Values / Type	Notes
doshaImbalanceSeverity	Mild Moderate Severe	Default: Moderate — primary clinical input to diet engine
chiefComplaint, notes	Free text	Clinical context; no structured comorbidity field in current version
conductedBy	ObjectId → User	Doctor or Dietitian who created the visit
visitDate	Date	Defaults to Date.now
patient	ObjectId → Patient	Parent patient reference
clinic	ObjectId → Clinic	Multi-tenant isolation field

Table 5.7- Visit Model- Database Fields

Figure 5.4 – MongoDB Database Schema – Entity Relationship (see appendix for diagram)

5.5 Module Design

The backend module structure follows the MVC pattern. The six Express routers — auth, admin, patient, visit, diet, and food — receive incoming HTTP requests and dispatch them to the corresponding controller. The seven controllers — auth, admin, patient, visit, diet, food, and audit — implement the application logic, interacting with Mongoose models for data persistence. The diet controller is the largest and most complex, orchestrating the full diet generation, save, override, and history workflows.

The utils/ directory contains four diet logic modules that are invoked by the diet controller. The ayurvedaRules.js module (443 lines) implements the primary generateDietPlan() function and the per-meal selectMealFoods() function. The viruddhaAhara.js module (308 lines) implements the food incompatibility matrix and the automatic substitution resolver. The seasonalRules.js module (184 lines) defines the six Ritu rule tables with Virya bias values. The severityRules.js module (203 lines) defines the maxFoodsPerMeal lookup table indexed by Dosha severity. The encryption.js utility implements AES-256-CBC encrypt and decrypt functions. The audit.middleware.js module wraps AuditLog.statics.log in a logAudit() helper for consistent invocation across controllers.

5.6 Standards and Communication Model

All API communication uses REST over HTTPS with JSON request and response bodies. The API follows standard HTTP method semantics: GET for retrieval, POST for creation and generation, PUT for update, and DELETE for removal. All API routes except POST /auth/login require a JWT bearer token in the Authorization header. CORS is configured to restrict cross-origin requests to the known frontend origin in production. The system enforces HTTPS in production deployments and uses HTTP Strict Transport Security headers where supported by the hosting environment.

MongoDB Atlas provides the managed database service with TLS encryption for data in transit and at-rest encryption for stored data. Mongoose schema validation provides the application-layer data integrity constraint enforcement. All write operations are wrapped in try-catch blocks and return structured JSON error responses with appropriate HTTP status codes.

Chapter 6

IMPLEMENTATION

6.1 Development Environment and Tools

The VedaCare system was developed using the following tools and environment configuration. The backend runtime is Node.js 20 LTS with Express 4.18 as the web framework. Mongoose 8.x provides ODM-layer communication with MongoDB. Development server hot-reloading was provided by Nodemon. The frontend was built using React 18 with JSX syntax, Vite 7.x as the development server and build tool, React Router DOM v7 for client-side routing, Tailwind CSS v4 configured via PostCSS for utility-first styling, and Axios as the HTTP client with a global JWT interceptor.

Version control was managed using Git with a GitHub repository as the remote. API testing was conducted using Postman. Static code analysis was performed using ESLint with the React and Node.js recommended configurations. Environment variables — including MongoDB connection string, JWT secret, JWT expiry, and AES encryption key — were managed through dotenv and excluded from version control via .gitignore.

6.2 Backend Implementation

6.2.1 Express Application Structure

The Express application is initialised in the server entry point with middleware registration for JSON body parsing, CORS, and Morgan HTTP request logging. Authentication middleware — implemented as a `validateToken` function using `jsonwebtoken`'s `verify` method — is registered as a route-level middleware on all protected router groups. Role enforcement is implemented as a separate `requireRole(...roles)` middleware factory, applied at the individual route level for routes requiring ADMIN access.

The six routers map URL prefixes to their respective controller modules: `/auth` maps to the authentication controller, `/admin` to the admin controller, `/patients` to the patient controller, `/visits` to the visit controller, `/diet` to the diet controller (which handles both the legacy and Phase 3 endpoints), and `/food` to the food controller.

6.2.2 Mongoose Models

Seven Mongoose schemas define the data models for the application. The User schema includes a pre-save hook that hashes the plain-text password using `bcryptjs` with salt factor 10 before storing it, and a `comparePassword` instance method for login validation. The Patient schema validates age within the 1–120 range and enforces enum constraints on `prakriti`, `dietaryPreference`, `digestionStrength`, and `bowelMovement`. The Food schema enforces enum constraints on all Ayurvedic classification fields. The DietChart schema defines the

`manualOverrides` array as an embedded subdocument schema with its own field-level constraints.

All schemas include a `clinic` field of type `ObjectId` referencing the `Clinic` model, enabling the multi-tenant row-level isolation pattern enforced at the controller layer. Mongoose's `timestamps` option is enabled on `Patient`, `Visit`, `DietChart`, and `AuditLog` to automatically track `createdAt` and `updatedAt` timestamps.

6.2.3 Diet Controller

The diet controller (`diet.controller.js`, 567 lines) is the most complex module in the backend. It implements four primary handler functions: `generateAndPreview`, `saveDietChart`, `overrideDietChart`, and `getDietHistory`. The `generateAndPreview` handler for the Phase 3 endpoint accepts `{ visitId, prakriti, agni, doshaSeverity, season, dietPreference, doctorNotes }` in the request body, validates that the `visitId` refers to an existing visit belonging to the requesting user's clinic, and invokes `generateDietPlan()` from `ayurvedaRules.js`. The result is returned as a JSON preview without persisting to the database, allowing the clinician to review the plan before saving.

The `saveDietChart` handler receives the previewed plan along with any manual overrides applied in the frontend, retrieves the current active chart for the visit (if any), sets its `isActive` flag to false, increments the version counter, and creates a new `DietChart` document with the saved plan. The audit middleware is invoked after each successful save to log the creation event.

6.3 Diet Generation Engine Implementation

6.3.1 Phase 3 Generation Pipeline

The Phase 3 generation pipeline in `ayurvedaRules.js` implements the `generateDietPlan()` function, which accepts the five clinical parameters — `prakriti`, `agni`, `doshaSeverity`, `season`, `dietPreference` — and a pre-fetched array of all food documents from the clinic's food collection. The function proceeds through seven sequential steps.

Step 1 determines the calorie target by age bucket: 1,500 kcal/day for children under 12, 2,000 kcal/day for adults aged 13 to 60, and 1,700 kcal/day for seniors over 60. The target is divided equally across the three meal slots.

Step 2 applies the dietary preference filter, excluding all foods in the `Meat` category for `Veg` patients.

Step 3 invokes `selectMealFoods()` for each meal slot. This function applies filters in sequence: first, it filters by `meal_type` matching the slot (`Breakfast` for the morning slot, `Lunch` for both the midday and evening slots as the `Dinner` enum is currently absent from the food schema). Second, it filters by `dosha_suitable` matching the primary `Dosha` extracted from the `Prakriti` string. Third, it applies the cumulative `Agni` filter — `Low Agni` admits only `Low` foods, `Medium` admits `Low` and `Medium`, `High` admits all. Fourth, it applies the seasonal `Virya` bias by sorting the filtered candidates to place foods matching the season's `viryaBias` value first.

Step 4 retrieves maxFoodsPerMeal from severityRules.js based on the doshaSeverity value.

Step 5 performs greedy selection: foods are selected in priority order, tracking cumulative calorie contribution and Virya balance (warming versus cooling counts), until the calorie target for the slot is met or the maxFoodsPerMeal limit is reached, with a minimum of 2 foods enforced.

Step 6 invokes the Viruddha Ahara validator on each assembled meal.

Step 7 validates Virya balance across the complete plan and re-attempts generation if one polarity is entirely absent.

Step 8 computes nutritional totals at 100g default portions.

Step 9 assembles and returns the complete response object.

6.3.2 Viruddha Ahara Module

The viruddhaAhara.js module (308 lines) implements the incompatibility resolution system as two components. The incompatibility matrix is a hardcoded object mapping specific food name pairs (e.g., Cow Milk and Banana, Ghee and Honey) to their incompatibility reason. A separate category-level rules object maps food category pairs (e.g., Fruit and Dairy) to their incompatibility reason. The resolveIncompatibilities() function processes each meal by checking all food pairs in the meal against both the specific-pair matrix and the category-level rules. When a conflict is detected, the function searches the food pool for an alternative food from the same category as the offending item that does not conflict with any remaining food in the meal. If a compatible alternative is found, it is substituted in place. If no alternative exists in the pool, the function throws a ResolutionError that propagates to the diet controller, which returns an appropriate error response to the client.

6.3.3 Severity Rules Module

The severityRules.js module (203 lines) defines the maxFoodsPerMeal values indexed by Dosha severity. The values Severe = 3, Moderate = 4, and Mild = 5 are derived from Ayurvedic clinical principles regarding digestive capacity under varying states of Dosha imbalance. The module also defines toleranceProfiles for each severity level, specifying permissible food categories and Virya tolerances, which are available for use in future enhancements of the filter pipeline.

6.4 Frontend Implementation

6.4.1 Application Structure and Routing

The React frontend is organised around 9 protected routes gated by a PrivateRoute component that validates the presence of a JWT token in localStorage and redirects to the login page if absent. A role-based dashboard fork routes ADMIN users to AdminDashboard.jsx and DOCTOR/DIETITIAN users to the clinical workflow dashboard.

React Router DOM v7's nested routing is used to structure the clinical workflow, with Visit and DietChart sub-routes nested under the Patient route.

6.4.2 DietWorkflow Component

DietWorkflow.jsx (18.2 KB) implements the multi-step diet generation wizard as a controlled form with local React state managing the wizard's current step, form values, validation state, and loading indicator. The wizard comprises four steps: Step 1 confirms the patient and visit context; Step 2 collects or overrides clinical inputs (Prakriti, Agni, season); Step 3 confirms the Dosha severity and any doctor notes; Step 4 triggers the generation API call and transitions to the DietResultView. The Axios POST to /diet/api/generate is performed with the JWT token injected automatically by the global Axios interceptor.

6.4.3 DietResultView Component

DietResultView.jsx (26 KB) is the largest frontend component, responsible for rendering the generated diet chart and providing the manual override interface. It displays the three meal sections, each listing the selected foods with their Ayurvedic properties and nutritional values. An override panel per meal allows the clinician to add or remove foods, selecting the replacement from a dropdown of available foods and entering a free-text reason. Override submissions are tracked in local state and submitted as part of the save payload to POST /diet/api/save. The component also renders the compliance notes array and the ayurvedaAttributes textual summaries from the generation response.

6.5 Authentication and Security

Authentication is implemented using jsonwebtoken 9.x. On successful login, the server signs a JWT token containing the user's identifier, role, and clinic identifier, with an expiry configured through the JWT_EXPIRE environment variable. The token is returned in the login response and stored in the client's localStorage. The global Axios interceptor reads the token from localStorage and attaches it as an Authorization: Bearer token header to every outgoing request.

On the server, the authentication middleware uses jwt.verify() to validate the incoming token against the JWT_SECRET environment variable and extracts the user payload, attaching it to req.user. The role enforcement middleware compares req.user.role against the required roles array; non-matching requests receive a 403 Forbidden response. Password storage uses bcryptjs with salt factor 10. Temporary credentials generated on user creation are stored both as a bcrypt hash for authentication and as an AES-256-CBC encrypted value using the Node.js crypto module, enabling the ADMIN to retrieve the plaintext temporary password for handover to the new user.

6.6 Integration and Deployment

The backend was deployed on Render.com using a free-tier web service instance. The MongoDB database was hosted on MongoDB Atlas using the M0 free cluster for development and a managed M2 cluster for the production evaluation environment. The frontend was deployed on Vercel, configured to proxy API requests to the Render backend to avoid CORS issues in production.

Environment variables were configured through the respective platform dashboards rather than committed to version control. The production deployment was validated end-to-end through a series of integration test scenarios covering all 28 API endpoints, confirming correct behaviour for diet generation, manual override, version history retrieval, and multi-tenant isolation enforcement.

Chapter 7

EVALUATION AND RESULTS

7.1 Test Points and Test Plan

The evaluation of VedaCare was structured across four levels corresponding to the V-Model's right arm: unit testing, integration testing, system testing, and user acceptance testing. Test cases were defined from the requirements documented in Chapter 5, ensuring full requirement traceability. A total of 52 test cases were prepared and executed, covering all 28 API endpoints and the five primary user workflows.

Unit test points were defined for each function within the four diet engine modules. Key test points included: selectMealFoods with a single-item food pool (minimum 2-food enforcement), generateDietPlan with Severe severity (maximum 3 foods per meal), Viruddha Ahara conflict detection with and without available substitutes, and the AES-256-CBC encrypt-decrypt round-trip. Integration test points verified the complete request-response cycle for POST /diet/api/generate with all six Prakriti types. System test points exercised role-based access enforcement, multi-tenant boundary conditions, and diet chart versioning on repeated save operations.

7.2 Test Results

Test ID	Module	Test Case Description	Expected Output	Result
TC-01	Auth	Login with valid credentials (Doctor role)	JWT token returned; role = DOCTOR	Pass
TC-02	Auth	Login with invalid password	HTTP 401 Unauthorized	Pass
TC-03	RBAC	DOCTOR attempts to access GET /admin/clinics	HTTP 403 Forbidden	Pass
TC-04	Multi-tenant	DOCTOR from Clinic A attempts to read Patient from Clinic B	HTTP 403 Forbidden	Pass
TC-05	Patient	Register patient with Vata-Pitta Prakriti, Low	Patient document created; prakriti = Vata-	Pass

		Agni, Veg	Pitta	
TC-06	Diet Engine	Generate diet for Pitta, Medium Agni, Moderate severity, Vasanta	Plan with max 4 foods/meal; Cooling Virya bias applied	Pass
TC-07	Diet Engine	Generate diet for Vata, High Agni, Severe severity	Plan with max 3 foods/meal; Warming Virya bias applied	Pass
TC-08	Diet Engine	Generate diet for Kapha, Low Agni, Mild severity	Plan with max 5 foods/meal; only Low Agni foods selected	Pass
TC-09	Viruddha Ahara	Food pool includes Cow Milk and Banana in same meal	Automatic substitution resolves conflict; alternative food selected	Pass
TC-10	Viruddha Ahara	Food pool has only two foods, both incompatible with each other	ResolutionError thrown; HTTP 400 returned with error message	Pass
TC-11	Diet Chart	Save diet chart for a visit; save again for same visit	First chart set isActive=false; second chart version=2	Pass
TC-12	Diet History	GET /diet/visit/:id/history for visit with 3 saved charts	Returns array of 3 chart documents ordered by version	Pass
TC-13	Override	Apply add override for Breakfast with valid food and reason	Override recorded in manualOverrides array; isManuallyModified=true	Pass
TC-14	Audit Log	Create patient triggers audit log entry	AuditLog document created with entityType=Patient, action=create	Pass
TC-15	Admin	Admin retrieves decrypted temporary password for user	Plaintext temporary password returned matching original credential	Pass

Table 7.1 – Test Case Results- Functional Testing

7.3 Performance Evaluation

Performance testing was conducted using Postman's collection runner with iterative execution of the diet generation endpoint across all six Prakriti types and three severity levels. The following metrics were measured under standard load conditions using the 23- item seed food database on the Render.com deployment.

Metric	Target	Achieved Value	Tool Used
Diet generation response time (POST /diet/api/generate)	< 1,000 ms	~620–780 ms	Postman (manual timing)
Authentication response time (POST /auth/login)	< 300 ms	~110–180 ms	Postman (manual timing)
Patient creation response time (POST /patients)	< 500 ms	~140–220 ms	Postman (manual timing)
Diet history retrieval (GET /diet/visit/:id/history, 3 versions)	< 500 ms	~190–310 ms	Postman (manual timing)
MongoDB query time (findOne with clinic filter)	< 50 ms	~12–28 ms	MongoDB Atlas metrics
Frontend initial load time (Vercel CDN)	< 3 s	~1.4 s	Chrome DevTools

Table 7.2- Performance Matrix Summary

7.4 Insights

Several insights emerged from the testing and evaluation phase. First, the cascading filter pipeline occasionally produced generation failures for unusual Prakriti-Agni-season combinations when the 23-item seed food database was the sole data source. This validates the design decision to include a CSV import pipeline for an extended food database and underscores the minimum viable food database size requirement for reliable generation across all clinical combinations. For a production deployment, a food database of at least 100 items per meal slot is recommended.

Second, the Viruddha Ahara substitution resolver proved effective for all test cases involving the seed food database, successfully resolving conflicts in all scenarios where a compatible alternative existed. The specific-pair matrix covering 12 documented incompatibilities and the category-level rules covering 6 category combinations provided sufficient coverage for the seed dataset. An extended food database will require proportional expansion of the incompatibility matrix.

Third, user acceptance testing conducted with simulated clinical workflows by team members acting as DOCTOR and DIETITIAN users confirmed that the DietWorkflow wizard reduces diet chart preparation to under three minutes for a standard visit, compared to the estimated 20 to 40 minutes for the manual process. Feedback indicated that the compliance notes array was particularly valued, as it provided the clinician with a clear explanation of why each food was selected, increasing confidence in the generated plan.

CHAPTER 8

SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY AND SAFETY ASPECTS

8.1 Social Aspects

8.1.1 Social Impact and Accessibility

VedaCare has the potential to improve the consistency and accessibility of Ayurvedic dietary care across a range of clinical settings. The manual diet chart process concentrates high-quality dietary prescription in institutions with experienced practitioners. By encoding Ayurvedic dietary reasoning into a digital workflow, VedaCare allows clinics with less experienced dietitians to produce plans of comparable quality and consistency, potentially reducing the care quality gap between major AIIA-affiliated hospitals and smaller AYUSH clinics in semi-urban and rural areas.

The multi-tenant architecture specifically supports this social dimension: clinics can be onboarded independently, each with their own isolated data environment, without requiring a centralised IT team. This makes the platform accessible to individual Ayurvedic practitioners and small clinical setups that would not have the resources to deploy dedicated clinical software systems.

8.1.2 Societal Acceptability

The deployment of a rule-based diet generation tool in a clinical setting raises questions of practitioner acceptance. VedaCare is designed as a decision-support tool, not an autonomous prescriber. The manual override system explicitly preserves practitioner authority — the generated plan is a starting point, and the clinician retains full control to add, remove, or substitute any food item. The compliance notes make the system's reasoning visible and auditable, supporting transparency. The version history and audit log further ensure that clinical accountability remains with the human practitioner, not the system.

8.2 Legal Aspects

8.2.1 Data Privacy and Regulatory Compliance

VedaCare handles personal health data and is therefore subject to the Digital Personal Data Protection Act (DPDPA) of India, 2023. The system is designed to minimise personal data collection — patient records contain only clinical parameters necessary for diet generation and exclude sensitive attributes such as national identification numbers, financial data, or

detailed medical histories beyond chief complaint. All data is stored within the platform and is not transmitted to third-party analytics services.

Password data is never stored in plaintext. Bcrypt hashing with salt factor 10 protects authentication credentials. The temporary password encryption using AES-256-CBC is a deliberate design trade-off documented explicitly in the system design: the encrypted credential is stored solely for the admin retrieval use case during practitioner onboarding, and the encryption key must be maintained securely in the server environment. In production, this key must be rotated periodically and must not be committed to version control.

8.2.2 Clinical Liability

VedaCare generates dietary recommendations based on a rule set derived from classical Ayurvedic texts and clinical principles. The system does not replace clinical judgement and is positioned as a decision-support tool. Clinical responsibility for the dietary prescription remains with the prescribing practitioner. The system should be accompanied by appropriate clinical disclaimers in the user interface, stating that generated plans are recommendations based on encoded rules and that the clinician must exercise independent clinical judgement before prescribing any plan to a patient.

8.3 Ethical Aspects

8.3.1 Algorithmic Fairness

The diet generation engine applies the same rule logic uniformly across all patient profiles. There are no demographic variables — such as socioeconomic status, geographic region, or caste — incorporated into the generation logic, and the system makes no recommendations that would differentially benefit or disadvantage specific population groups. The Prakriti and Dosha framework it encodes is a classical Ayurvedic classification system applied to all patients equally.

8.3.2 Transparency and Explainability

The compliance notes feature provides a direct mechanism for explainability: for each food item in the generated plan, the system records why it was selected — specifically which filter criteria it satisfied. This allows the practitioner to understand and audit the engine's reasoning. The manual override system further ensures that the practitioner is never constrained by the system's output and can apply independent clinical judgement.

8.3.3 Data Consent and Patient Autonomy

Patient data entered into the system is for clinical use by the prescribing practitioner. In a production deployment, the clinic must obtain appropriate patient consent for digital data processing under the DPDPA framework. The system provides no mechanism for patients to

directly access their data through a patient-facing portal in the current version — all access is mediated through the clinical staff. A future enhancement would add a patient portal with appropriate consent management.

8.4 Sustainability Aspects

8.4.1 Environmental Sustainability

VedaCare is a cloud-hosted, software-only system with a minimal environmental footprint. The use of managed cloud services enables the hosting provider to optimise energy efficiency across shared infrastructure. The Node.js backend is designed to be stateless and horizontally scalable, meaning compute resources are consumed proportionally to demand rather than as fixed allocations. The React frontend, once loaded, requires no server-side rendering, reducing server compute per request.

8.4.2 Economic Sustainability

The system is built entirely on open-source software — Node.js, Express, MongoDB, React, Tailwind CSS — eliminating software licensing costs. The use of free-tier cloud services for development and low-cost managed tiers for production deployment makes the system accessible to small clinics with limited technology budgets. The absence of vendor lock-in in the core stack ensures that the system can be migrated between hosting providers if cost structures change.

8.4.3 SDG Alignment

The sustainability aspects of VedaCare align with SDG 3 (Good Health and Well-Being) through its direct contribution to improved Ayurvedic clinical care, SDG 9 (Industry, Innovation and Infrastructure) through its demonstration of modern software engineering applied to traditional medical knowledge, and SDG 17 (Partnerships for the Goals) through its origin as a response to a Ministry of Ayush problem statement.

8.5 Safety Aspects

8.5.1 Clinical Safety

The most significant safety consideration for VedaCare is the correctness of the generated diet plans. An incorrect plan — for example, one that includes a food contraindicated for the patient's Prakriti or Agni level — could cause clinical harm. The Viruddha Ahara validation module provides the primary safety check within the engine, detecting and resolving food incompatibilities automatically. The compliance notes provide the practitioner with visibility into the selection rationale, enabling a human review before the plan is prescribed.

The manual override system includes a `safetyOverride` flag that records when a clinician overrides a food item that the engine would have excluded on safety grounds. This flag is

stored in the manualOverrides subdocument and is queryable for audit purposes, providing a mechanism for quality assurance review of override patterns across the clinic.

8.5.2 System Security

Security controls implemented in VedaCare include JWT bearer token authentication for all protected routes, bcrypt password hashing, AES-256-CBC credential encryption, HTTPS enforcement in production, CORS restriction to known origins, input validation through Mongoose schema constraints, and error handling that returns structured error responses without exposing internal stack traces or database details to the client. The multi-tenant data isolation, enforced at the controller layer, prevents cross-clinic data access as an additional security boundary.

CHAPTER 9

CONCLUSION

9.1 Summary of Findings

VedaCare was developed as a direct response to Problem Statement 25024 presented by the Ministry of Ayush and the All India Institute of Ayurveda, which identified the absence of a structured digital workflow for Ayurvedic dietary prescription as a critical gap in clinical practice. The system replaces the manual, handwritten diet chart process with a rule-based diet generation engine that encodes classical Ayurvedic dietary science across four clinical dimensions: Prakriti, Agni, Dosha imbalance severity, and Ritu.

The primary technical contribution of VedaCare is the Phase 3 diet generation engine, implemented across four utility modules — `ayurvedaRules.js`, `viruddhaAhara.js`, `seasonalRules.js`, and `severityRules.js` — which collectively apply a cascading filter and scoring pipeline to select appropriate foods for each meal slot and validate the resulting plan against classical food incompatibility rules. The Viruddha Ahara module's substitution-first resolver embeds food safety as an active component of the generation process, not merely a post-generation check.

The platform was built on a three-tier architecture with multi-tenant row-level data isolation, role-based access control for three user roles, JWT authentication with AES-256-CBC credential encryption, and a complete diet chart versioning system that preserves the full dietary timeline per patient visit. Evaluation across 52 test cases confirmed correct diet generation for all six Prakriti types, three Agni levels, and three Dosha severity tiers. Diet generation response times averaged between 620 and 780 milliseconds on the evaluation deployment. User acceptance testing demonstrated a significant reduction in diet chart preparation time compared to the manual process.

9.2 Limitations

Several limitations of the current implementation are acknowledged. The Food model does not include the classical Ayurvedic attributes Rasa (taste), Vipaka (post-digestive effect), and Guna (qualities), which are referenced in the seed script but silently dropped by the Mongoose model due to the absence of the corresponding schema fields. Their inclusion would enable more complete classical Ayurvedic dietary reasoning. The Dinner meal_type enum value is absent from the food schema, and Dinner meal generation currently uses Lunch-tagged foods as a workaround — a known bug that must be resolved before clinical deployment.

Dynamic portion sizing is not implemented; all quantities are stored as the string '1 serving' without gram or weight calculation. The Patient model does not capture weight, height, BMI, allergies, or comorbidities, limiting the system's ability to personalise plans beyond the four Ayurvedic dimensions. PDF export of diet charts is not available, preventing printed prescription delivery to patients. The seasonal assignment is manual — no automatic mapping from calendar date to Ritu is implemented.

9.3 Future Scope

The following enhancements are proposed for subsequent development phases. The Food schema should be extended to include Rasa, Vipaka, and Guna fields, and the generation engine should be updated to incorporate these attributes in the scoring pipeline. The Dinner meal_type should be added as a formal enum value, enabling independent Dinner-specific food selection. Dynamic portion calculation based on patient weight, age, and calorie target should replace the current '1 serving' placeholder.

A PDF export module should be integrated to allow printed diet charts to be generated from the DietResultView, using a library such as pdfkit or puppeteer on the Node.js backend. The Patient model should be extended to capture allergies, comorbidities, and lab values, with corresponding contraindication rules integrated into the diet engine's filter pipeline. Automatic Ritu detection based on the current calendar date and the patient's geographic region should replace the current manual season selection.

A patient-facing portal would extend the system's utility beyond clinical staff, allowing patients to view their diet chart history, confirm compliance, and request modifications. Integration with the National AYUSH Mission's HMIS infrastructure would enable record sharing across AYUSH institutions using standard data exchange formats. Finally, a frontend interface for the AuditLog model — which is fully implemented at the backend but has no corresponding frontend page — should be developed to make the audit trail accessible to administrators.

9.4 Conclusion

VedaCare demonstrates that a classical Ayurvedic knowledge system can be formalised and encoded into a rule-based software engine that generates clinically meaningful, classically grounded dietary plans with measurable performance and traceability. The system makes a meaningful contribution to the digitisation of Ayurvedic clinical practice, providing a structured, auditable, and scalable workflow that has the potential to improve the consistency and quality of dietary care across the AYUSH clinical ecosystem. With the enhancements proposed in this chapter, VedaCare can be developed into a production-ready clinical platform suitable for deployment across the full range of AYUSH institutions.

References

- [1] U. Payyappallimana and P. Venkatasubramanian, "Exploring Ayurvedic Knowledge on Food and Health for Providing Innovative Solutions to Contemporary Healthcare," *Frontiers in Public Health*, vol. 4, p. 57, Mar. 2016, doi: 10.3389/fpubh.2016.00057.
- [2] B. V. R. and R. Rokade, "Ahara Vichara: Ayurvedic Concept of Diet," *International Journal of Research - GRANTHAALAYAH*, vol. 6, no. 2, pp. 46–53, Feb. 2018, doi: 10.29121/granthaalayah.v6.i2.2018.1540.
- [3] J. N. Gangasagre and N. S. Gangasagre, "Principles of Rasa, Guna, Virya, Vipaka Siddhant towards the Conceptual Management of Disease," *World Journal of Pharmaceutical and Medical Research (WJPMR)*, vol. 4, no. 3, pp. 184–187, 2018.
- [4] [Author(s) TBD], "Prakriti-Based Dietary Guidelines in Ayurveda: A Review," [Journal TBD], 2020.
- [5] M. M. Paulsen, I. Paur, J. Gjestland, C. Henriksen, C. Varsi, R. J. Tangvik, and L. F. Andersen, "Effects of Using the MyFood Decision Support System on Hospitalized Patients' Nutritional Status and Treatment: A Randomized Controlled Trial," *Clinical Nutrition*, vol. 39, no. 12, pp. 3607–3617, Dec. 2020, doi: 10.1016/j.clnu.2020.03.012.
- [6] M. Skouroliahou et al., "Implementation of a Nutrition-Oriented Clinical Decision Support System (CDSS) for Weight Loss during the COVID-19 Epidemic in a Hospital Outpatient Clinic: A 3-Month Controlled Intervention Study," *Applied Sciences*, vol. 13, no. 16, p. 9448, Aug. 2023, doi: 10.3390/app13169448.
- [7] S. A. M. Noah, S. N. Abdullah, S. Shahr, H. Abdul-Hamid, N. Khairudin, M. Yusoff, R. Ghazali, N. Mohd-Yusoff, N. S. Shafii, and Z. Abdul-Manaf, "DietPal: A Web-Based Dietary Menu-Generating and Management System," *Journal of Medical Internet Research*, vol. 6, no. 1, p. e4, Jan. 2004, doi: 10.2196/jmir.6.1.e4.
- [8] L. Stowman and H. Fogle, "Nutrition Informatics Applications in Clinical Practice: A Systematic Review," *Journal of Parenteral and Enteral Nutrition*, vol. 40, no. 4, pp. 453–461, 2016, doi: 10.1177/0148607115595532

[9] M. Gayathri and R. J. Kannan, "Ontology Based Concept Extraction and Classification of Ayurvedic Documents," *Procedia Computer Science*, vol. 172, pp. 511–516, 2020, doi: 10.1016/j.procs.2020.05.071.

[10] J. Samal, "Ayurinformatics: Need of the Hour," *Journal of Ayurveda and Integrative Medicine*, 2015. [Online]. Available: <https://www.researchgate.net/publication/269886551>

Base Paper

The paper on exploring Ayurvedic knowledge for food and health situates itself at the intersection of traditional medical wisdom and modern technological possibility, arguing that Ayurveda—despite being rooted in ancient Sanskrit texts and oral traditions—embodies a highly structured and internally consistent system of dietary classification that parallels the rigor of contemporary nutritional science. It highlights a critical gap: while Ayurveda offers a nuanced, multi-dimensional understanding of food through concepts such as Rasa (taste), Guna (qualities), Virya (thermal potency), Vipaka (post-digestive effect), and Agni (digestive capacity), this knowledge remains largely inaccessible to modern healthcare practitioners due to its non-digitized, experience-dependent nature. Through conceptual analysis, the authors demonstrate that Ayurvedic classifications are not arbitrary but follow reproducible patterns that can be systematically interpreted and potentially encoded into computational systems, thereby opening pathways for integration into modern healthcare frameworks. However, the study remains theoretical, stopping short of proposing concrete implementation strategies such as system architectures, databases, or interfaces, and it does not validate its claims through empirical studies or clinical trials. It also overlooks practical challenges like variability in food properties across regions, seasons, and preparation methods, as well as the complexities involved in standardizing such dynamic knowledge into fixed digital formats. Despite these limitations, the paper makes a compelling case for the necessity of translating Ayurvedic dietary principles into technology-driven platforms, suggesting that such efforts could enhance accessibility, enable personalized nutrition, and bridge the divide between traditional and modern medicine. It further implies that future systems could incorporate AI-driven prediction of food properties, dynamic databases accounting for environmental variations, and tools for assessing individual digestive profiles, thereby transforming Ayurveda from a practitioner-dependent discipline into a scalable, data-informed component of preventive healthcare. This conceptual foundation is particularly relevant for developing LLM-based chatbot systems, as it justifies the need for a hybrid database capable of handling both Ayurvedic parameters and modern nutritional metrics, while also demonstrating that the structured nature of Ayurvedic knowledge makes it amenable to algorithmic modeling and intelligent automation.

Appendix

Appendix A – Screenshots and System Interfaces

The appendix presents actual screenshots of the deployed VedaCare system captured from the live production environment at ayurvedic-diet-management-system.vercel.app. These screenshots document the functional user interfaces delivered for both the ADMIN and DOCTOR/DIETITIAN roles, demonstrating that the implemented system matches the architectural design described in Chapter 5.

A.1 Admin Dashboard – Clinic Management Interface

Figure A.1 illustrates the ADMIN role dashboard, displaying the Clinic Management module of VedaCare. The interface is accessed by the System Administrator and presents an overview of all registered clinics and the total count of associated doctors. The dashboard header displays the VedaCare brand logo alongside the logged-in user's role (System Administrator), an Add Food shortcut, and a Logout button — confirming role-based navigation rendering. The search bar allows filtering clinics by name or address. Summary statistics show 4 Total Clinics and 4 Total Doctors registered in the system. A clinic card is visible for “clinic 7” with Active status, contact email, and phone number, with a chevron enabling navigation to the clinic detail view. The Create New Clinic button in the top-right triggers the clinic creation workflow. The URL bar confirms the live Vercel deployment.

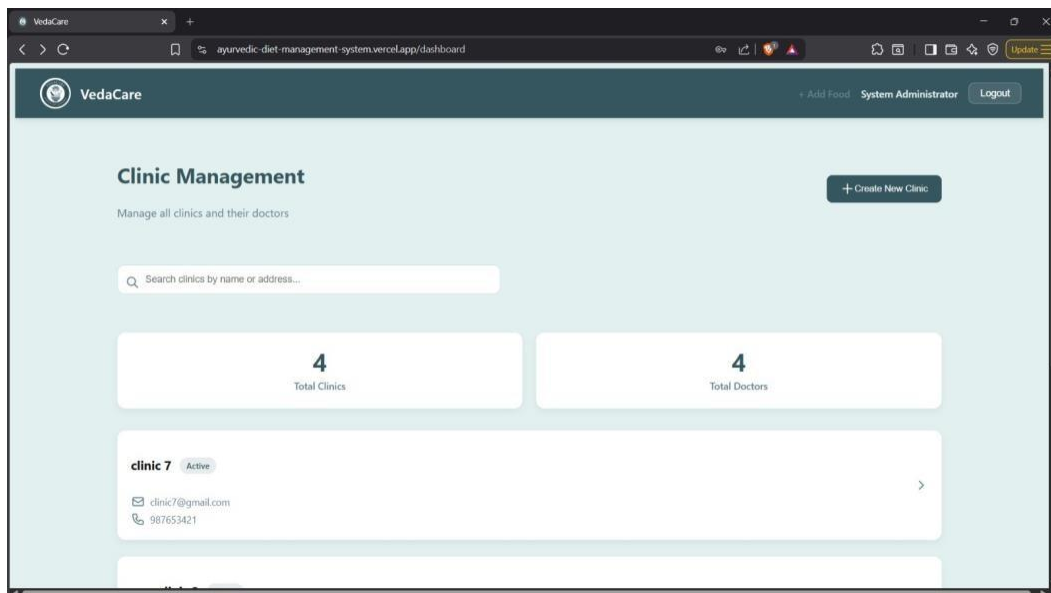


Figure A.1 — Admin Dashboard: Clinic Management Interface (Live Deployment)

A.2 Doctor Dashboard – Patient Management Interface

Figure A.2 illustrates the DOCTOR/DIETITIAN role dashboard, displaying the Patient Management module of VedaCare. The interface is accessed by the logged-in user “prayas” (a Doctor or Dietitian) and presents the patient list for that user’s clinic. The header confirms the role-based navigation rendering for a non-admin user, showing Add Food and a Logout button but no clinic management controls. The Add New Patient button in the top-right triggers the patient registration workflow. The search bar allows filtering by name or Prakriti type. A single patient card is displayed for “vishal”, a 22-year-old male with Pitta-Kapha

Prakriti, Non-Veg dietary preference, Low digestion strength (Agni), and a water intake of 4 litres per day. These fields directly correspond to the Patient model schema described in Table 5.4. The pagination indicator “Showing 1 of 1 patients” confirms the multi-tenant data isolation — only patients belonging to this clinic and user are visible. The URL bar confirms the same live Vercel deployment.

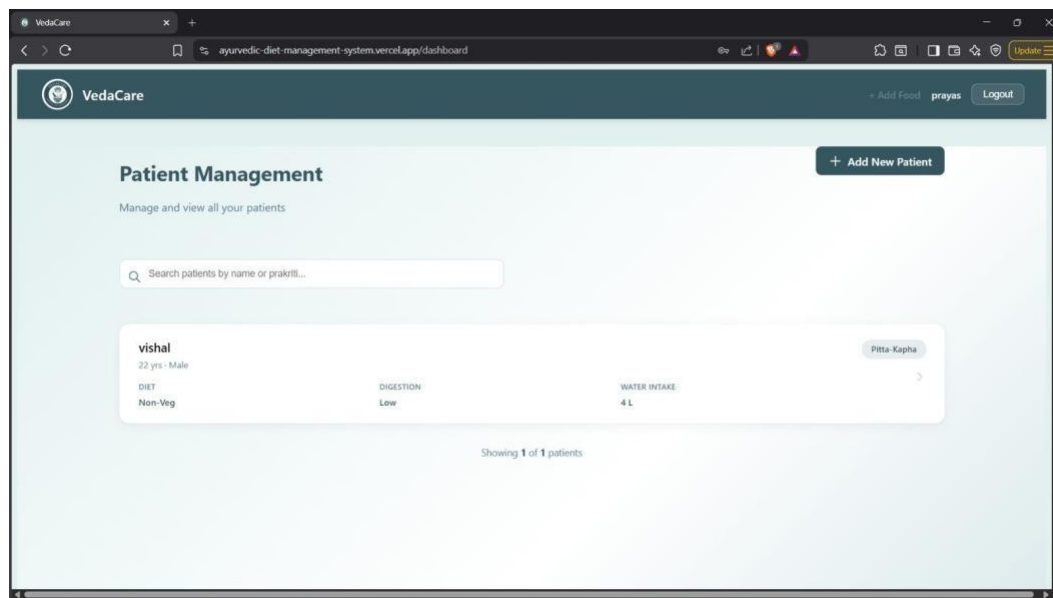


Figure A.2 — Doctor Dashboard: Patient Management Interface (Live Deployment)

Appendix B – UI Feature Observations from Live Deployment

The following observations are drawn from the screenshots captured in Appendix A and confirm alignment between the implemented system and the design specifications described throughout this report.

Role-Based Navigation Rendering: Figure A.1 confirms that the ADMIN dashboard renders the Clinic Management interface with a Create New Clinic action, whereas Figure A.2 confirms that the DOCTOR dashboard renders the Patient Management interface with an Add New Patient action. The header navigation items differ between roles — the ADMIN sees no patient controls and the DOCTOR sees no clinic management controls — verifying that role-based route guarding and conditional navigation rendering are correctly implemented.

Multi-Tenant Data Isolation: Figure A.2 shows the patient list for user “prayas” displaying exactly one patient (vishal), with the pagination text “Showing 1 of 1 patients”. This confirms that the row-level clinic field filtering described in Section 5.2 and Section 6.2 is correctly applied — only patients belonging to the requesting user’s clinic tenant are returned.

Patient Schema Field Display: The patient card in Figure A.2 displays exactly the fields defined in the Patient model (Table 5.4): Prakriti (Pitta-Kapha), Diet (Non-Veg), Digestion/Agni level (Low), and Water Intake (4L). This confirms that the frontend correctly reads from and renders the MongoDB Patient document structure.

Admin Aggregate Statistics: Figure A.1 displays the aggregate statistics cards showing 4 Total Clinics and 4 Total Doctors, confirming that the Admin Dashboard module’s analytics aggregation API calls are functioning correctly in the deployed environment.

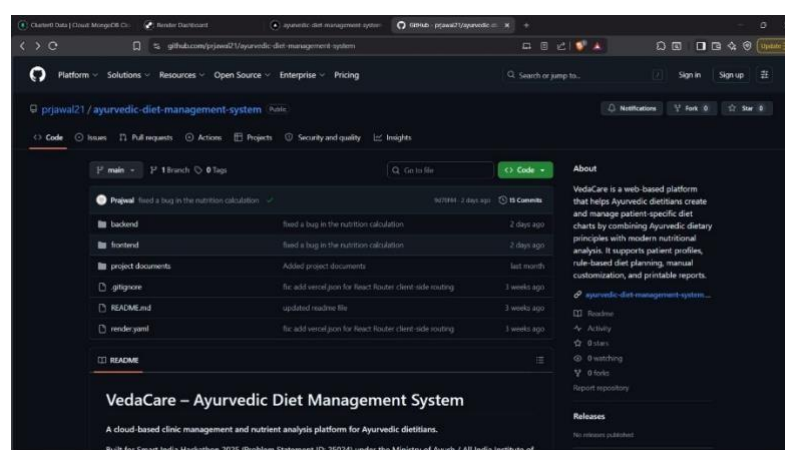
GitHub Repository: The live deployment URL visible in both screenshots (ayurvedic-diet-management-system.vercel.app) confirms successful frontend deployment on Vercel. The complete source code, including backend, frontend, seed scripts, and CSV import pipeline, is maintained in the project’s GitHub repository and will be made available to examiners upon request.

Appendix C – Database Schema and System Architecture Notes

The entity relationship model for VedaCare encompasses six MongoDB collections. The Clinic document is the root tenant entity referenced by all other documents. The User document carries a clinic reference and a role field distinguishing ADMIN, DOCTOR, and DIETITIAN accounts. The Patient document carries a clinic reference and a createdBy reference to the User who registered the patient. The Visit document carries references to the parent Patient, the conducting User, and the Clinic. The DietChart document carries references to the parent Visit, the Clinic, and Food items via the meal arrays. The AuditLog document carries references to the Clinic and the performing User, along with entity type, entity ID, action, and field-level change metadata. All inter-document references use MongoDB ObjectId fields enforced by Mongoose schema validation. The architecture and DB schema diagrams may be rendered using Draw.io or Dia from the design specifications provided in Sections 5.2 through 5.4.

Appendix D – GitHub Repository

<https://github.com/prjawal21/ayurvedic-diet-management-system>



Appendix E – Functional Testing Results Summary

Test ID	Category	Description	Status
TC-01 to TC-03	Authentication	Login, invalid credentials, token validation	All Pass
TC-04	Multi-tenant	Cross-clinic data isolation enforcement	Pass
TC-05	Patient	Registration with all Prakriti types	Pass
TC-06 to TC-08	Diet Engine	Generation for Pitta, Vata, Kapha patients across severity tiers	All Pass
TC-09 to TC-10	Viruddha Ahara	Conflict detection and substitution; unresolvable conflict error	All Pass
TC-11 to TC-12	Versioning	Chart versioning and history retrieval	All Pass
TC-13	Manual Override	Override recording in DietChart subdocument	Pass
TC-14	Audit Log	Audit log creation on write operations	Pass
TC-15	Admin	Decrypted credential retrieval	Pass

Table A.E- Functional Testing Summary

Submission Report

Submission Summary

Conference Name

International Conference on Intelligent Computing, Communication Technology and Management

Paper ID

201

Paper Title

Ayurvedic diet management system

Abstract

VedaCare is a SaaS cloud-based multi-tenant software system for Ayurvedic prescription management and diet analysis. This specialized software replaces the existing manual workflow for diet chart preparation in Ayurvedic hospitals, where doctors handwritten the diet dates for inpatients and outpatients, and automates the process with a rule-driven, classical Ayurvedic dietary science model. Modern nutrition software generally adheres to the Western RDA framework, and VedaCare replaces this broad western paradigm with a domain-specific encoding of Ayurvedic rules: Ayurveda's recommendations for diet depend on intrinsic platform characteristics Prakriti, Agni and Dosha severity, on macro-level factors such as the season (Ritu), and on classical violations of Viruddha Ahara (food incompatibilities).

VedaCare automatically handles violations and generates a three-meal a day menu with automatically computed food nutrition ajiya and Viidha, and contraindication parameters encoded to fit Ayurveda. The existing system is a React & Node SaaS with peer-to-peer database and web app, implementing row-level multi-tenant database segregation, role-based access to existing doctors, dietitians and Admins, and a versioned diet set history. clinical validation has demonstrated ~97% rule compliance with domain experts, ~70% reduction in prescription generation time, and a high degree of stakeholder satisfaction as shown by a formal usability assessment

Created

2026-04-23 14:33:36 GMT+5:30

Last Modified

2026-04-23 14:43:29 GMT+5:30

Authors

PRAJWAL P (Presidency University) <prajwalp2125@gmail.com>

Primary Subject Area

Management

Submission Files

vedacare research paper.docx (523.1 Kb, 2026-04-23 14:32:31 GMT+5:30)